

UNIVERSIDAD TECNOLÓGICA NACIONAL

FACULTAD REGIONAL MENDOZA

Tecnicatura Universitaria en Programación

**SISTEMA DE GESTIÓN DE STOCK INTEGRADO CON API REST Y
AUTOMATIZACIÓN DE PEDIDOS**

Trabajo Final para optar al título de Técnico Universitario en Programación

Autores: Juan Ignacio Juarez, Mariano Videla

Director: Mag. en Ing. de Software Alberto Cortez

Norma de citación: APA 7.^a edición

Mendoza – Argentina
2026

RESUMEN

En el contexto de la transformación digital de las pequeñas y medianas empresas (PyMEs), la gestión eficiente del inventario y la automatización de pedidos representan desafíos operativos críticos cuya resolución impacta directamente en la competitividad y la rentabilidad organizacional. El presente trabajo tuvo como propósito el diseño, desarrollo e implementación de un sistema integrado de gestión de stock basado en una arquitectura API REST. El sistema se orientó a centralizar la administración de productos, clientes y ventas, e incorporar mecanismos de automatización en la recepción y procesamiento de pedidos mediante plataformas de mensajería instantánea.

La investigación se enmarcó en el enfoque de Design Science Research (DSR), con un diseño pretest-posttest de un único grupo, ejecutado mediante un experimento controlado. La solución tecnológica se desarrolló con Java para el backend, JavaScript para el frontend, MySQL como sistema gestor de base de datos relacional y n8n como herramienta de automatización de flujos. Esta última se integró con la API de Telegram para la gestión automatizada de pedidos.

Los resultados se obtuvieron sobre 20 escenarios de pedido estandarizados por condición que reproducen las condiciones operativas de un comercio minorista de pequeña escala. Mostraron una reducción del 34,9% en el tiempo medio de procesamiento de pedidos, con un tamaño del efecto grande (prueba de Wilcoxon, $p < 0,0001$; $r = 0,60$). La mediana de las diferencias pareadas fue de 11,50 s, equivalente al 35,9% de la mediana del pretest (IC95% de esa reducción relativa: [25,8%; 44,4%]). El resultado queda por debajo del umbral del 50% hipotetizado a priori, que ese intervalo excluye por completo, pero es estadísticamente significativo y sustantivamente relevante. Si se incluye el tiempo de un intento abortado que la medición principal deja fuera del intervalo, la reducción de las medias baja al 22–24% y sigue siendo significativa. La tasa de errores en la carga de datos y el nivel de actualización del stock no mostraron diferencias entre condiciones, resultado que se discute en el Capítulo 6 a la luz de una limitación reconocida del propio diseño experimental. Estos hallazgos ponen de manifiesto el potencial de la automatización como herramienta estratégica para la modernización de procesos comerciales en entornos de pequeña escala, con resultados más contundentes en la reducción de tiempos que en la exactitud de los datos registrados. La validación del sistema en un comercio real en operación queda planteada como línea de trabajo futuro prioritaria.

Palabras clave (español): gestión de inventario, API REST, automatización de procesos, mensajería instantánea, pequeñas y medianas empresas, Design Science Research, sistema de información.

Keywords (English): inventory management, REST API, process automation, instant messaging, small and medium enterprises, Design Science Research, information system.

ÍNDICE

CAPÍTULO 1: Introducción	5
1.1 Contexto del problema.....	5
1.2 Planteamiento del problema e hipótesis de trabajo	5
1.3 Justificación	6
1.4 Objetivo general	6
1.5 Objetivos específicos	7
1.6 Alcance y limitaciones	7
CAPÍTULO 2: Marco Teórico y Estado del Arte	9
2.1 Sistemas de gestión de inventario en PyMEs	9
2.2 Arquitectura API REST.....	9
2.3 Separación frontend y backend	10
2.4 Automatización de procesos de negocio.....	11
2.5 Integración con plataformas de mensajería instantánea	12
2.6 Bases de datos relacionales.....	13
2.7 Estado del arte y vacío de conocimiento.....	13
2.7.1 Sistemas de gestión para PyMEs.....	13
2.7.2 Automatización e integración de chatbots en sistemas de gestión.....	14
2.7.3 Vacío de conocimiento identificado.....	14
CAPÍTULO 3: Marco Metodológico.....	16
3.1 Tipo de investigación e hipótesis de trabajo	16
3.2 Paradigma epistemológico	16
3.3 Enfoque de investigación: Design Science Research (DSR)	17
3.4 Diseño de investigación	17
3.5 Variables de estudio y operacionalización.....	20
3.6 Técnicas e instrumentos de recolección de datos	21
3.7 Herramientas y entorno tecnológico.....	21
3.8 Procedimiento de investigación	22
3.9 Criterios de validez y confiabilidad	23
CAPÍTULO 4: Desarrollo del Sistema	24
4.1 Arquitectura del sistema	24
4.1.1 Implementación del frontend (SPA).....	25
4.2 Diseño de la API.....	27
4.3 Modelo de datos	28
4.4 Integración con Telegram y automatización con n8n.....	30
4.5 Pruebas del sistema y validación del artefacto	33

4.6 Evolución del artefacto durante y después del experimento	34
CAPÍTULO 5: Resultados.....	37
5.1 Resultados del experimento controlado	37
5.2 Síntesis de hallazgos	40
CAPÍTULO 6: Discusión.....	41
6.1 Interpretación de los resultados.....	41
6.2 Limitaciones del estudio	42
CAPÍTULO 7: Conclusiones y Trabajos Futuros	46
7.1 Conclusiones.....	46
7.2 Trabajos futuros.....	47
Referencias Bibliográficas	49
Anexos	52
Anexo A. Dataset crudo del experimento.....	52
Anexo B. Análisis estadístico.....	53
Anexo C. Flujo de automatización de n8n	53
Anexo D. Modelo de datos	54
Anexo E. Capturas de la interfaz (SPA) y del bot de Telegram	54
Anexo F. Repositorio de código fuente.....	54
Nota sobre herramientas de prueba de API.....	55

CAPÍTULO 1: Introducción

1.1 Contexto del problema

En las últimas décadas, la evolución de las tecnologías de la información ha generado profundas transformaciones en la forma en que las organizaciones gestionan sus procesos internos. En el ámbito de las pequeñas y medianas empresas (PyMEs), dichas transformaciones adquieren una relevancia particular, dado que la incorporación de herramientas tecnológicas puede representar una ventaja competitiva significativa frente a actores de mayor tamaño y recursos.

Uno de los procesos más críticos dentro de este tipo de organizaciones es la gestión del stock, que involucra el control de inventarios, el registro de ventas y la administración de clientes. Sin embargo, en numerosos casos estos procesos se siguen realizando mediante mecanismos manuales o sistemas parcialmente digitalizados y no integrados entre sí. Esto genera inconsistencias en la información, redundancia en la carga de datos y una capacidad de análisis limitada para la toma de decisiones.

A su vez, el crecimiento exponencial del uso de plataformas de mensajería instantánea como canal de comunicación comercial introduce nuevas oportunidades para la automatización de procesos. No obstante, la falta de integración entre estos canales y los sistemas de gestión existentes implica una sobrecarga operativa y una mayor probabilidad de errores humanos. Esta brecha tecnológica es especialmente pronunciada en comercios de pequeña escala, donde los recursos humanos y financieros destinados a la gestión de sistemas son limitados.

1.2 Planteamiento del problema e hipótesis de trabajo

La inexistencia de un sistema integrado que permita gestionar de manera unificada la información relacionada con productos, clientes, ventas y pedidos recibidos a través de plataformas digitales constituye una limitación estructural en la eficiencia operativa de los pequeños comercios. Esta situación se traduce en demoras en la atención al cliente, falta de actualización en tiempo real del stock disponible y dificultades en la toma de decisiones estratégicas basadas en datos confiables.

En este contexto, se formula la siguiente pregunta de investigación:

¿De qué manera la implementación de un sistema de gestión de stock basado en una arquitectura API REST, integrado con herramientas de automatización y plataformas de mensajería, contribuye a mejorar la eficiencia operativa y reducir errores en la gestión de pedidos en pequeños comercios?

Hipótesis de trabajo: H1: la implementación de un sistema integrado de gestión de stock basado en arquitectura API REST, con automatización de pedidos mediante WhatsApp y Telegram, reducirá el tiempo promedio de procesamiento de pedidos en al menos un 50%, en comparación con la modalidad manual simulada. Esta reducción se mide en un entorno de prueba controlado que reproduce las condiciones operativas típicas de un comercio minorista de pequeña escala. El umbral adoptado se fundamenta en los órdenes de magnitud de mejora reportados para la

automatización de procesos administrativos estructurados (Davenport y Kirby, 2016) y en los beneficios de eficiencia identificados para la adopción de tecnología en PyMEs (Moeuf et al., 2020). Esto no implica una derivación estadística directa de dichos antecedentes: se trata de un umbral de referencia que los autores adoptaron para volver la hipótesis falsable, no de una cifra reportada literalmente por esas fuentes. La tasa de errores en la carga de datos (TED) y el nivel de actualización del stock (NAS) se miden también (sección 3.5) como indicadores descriptivos secundarios del desempeño del sistema, sin un umbral de mejora fijado a priori. Como se explica en la sección 3.4, el diseño experimental —ejecutado por uno de los autores del sistema en ambas condiciones— no permite generar una línea de base de errores representativa de un operador real, por lo que una hipótesis formal sobre TED no sería falsable con este diseño. Esta limitación se discute en la sección 6.1 y su contrastación con operadores reales queda planteada como trabajo futuro (sección 7.2).

H1 es operacionalizable mediante la variable TPP definida en la sección 3.5; el diseño pretest-posttest descrito en la sección 3.4 permitió evaluarla. TED y NAS se miden con el mismo diseño, pero se reportan de forma descriptiva, por las razones ya señaladas.

1.3 Justificación

La relevancia del presente trabajo radica en la necesidad de aportar soluciones tecnológicas concretas a problemáticas reales del sector comercial, particularmente en organizaciones de pequeña escala que carecen de sistemas integrados de gestión. Según datos de la CEPAL, las mipymes representan el 99% de las empresas de la región y contribuyen con el 61% del empleo formal (CEPAL, 2019). Sin embargo, su nivel de adopción tecnológica sigue siendo significativamente inferior al de las grandes empresas (OECD, 2023).

Esta brecha de adopción tecnológica no es exclusiva de las economías en desarrollo. Pfister y Lehmann (2023) estudian PyMEs alemanas y encuentran que el tamaño de la empresa y la disponibilidad de recursos internos siguen siendo los principales condicionantes del éxito de los procesos de transformación digital, incluso en un contexto de alta disponibilidad tecnológica. Esto refuerza la pertinencia de desarrollar soluciones de bajo costo y fácil adopción, como la propuesta en este trabajo, específicamente pensadas para organizaciones con recursos técnicos limitados.

Desde una perspectiva práctica, el desarrollo de un sistema automatizado permite reducir la intervención manual en procesos críticos, mejorar la precisión de la información y optimizar los tiempos de respuesta. Desde el punto de vista académico, la investigación contribuye al análisis de la aplicación de arquitecturas REST y herramientas de automatización en entornos reales, generando conocimiento aplicable en el ámbito del desarrollo de software para PyMEs.

1.4 Objetivo general

Desarrollar e implementar un sistema integrado de gestión de stock basado en arquitectura API REST que permita optimizar la administración de productos, clientes y ventas mediante la automatización de pedidos a través de plataformas de mensajería instantánea. Evaluar,

además, su impacto sobre la eficiencia operativa mediante un experimento controlado que reproduce las condiciones operativas típicas de un comercio de pequeña escala.

1.5 Objetivos específicos

Los objetivos específicos del presente trabajo son:

1. OE1. Diseñar una arquitectura de software basada en el modelo cliente-servidor que separe de manera clara la capa de presentación y la capa de lógica de negocio.
2. OE2. Implementar una API REST utilizando el lenguaje de programación Java, con endpoints para las operaciones CRUD sobre las entidades centrales del dominio (productos, clientes, ventas).
3. OE3. Desarrollar una interfaz de usuario en JavaScript que facilite la interacción con el sistema para los operadores del comercio.
4. OE4. Integrar un bot de mensajería en la plataforma Telegram para la recepción automatizada de pedidos de clientes.
5. OE5. Implementar flujos de automatización mediante n8n que conecten el canal de mensajería con la API REST, eliminando la necesidad de intervención manual en el registro de pedidos.
6. OE6. Evaluar el desempeño del sistema desarrollado mediante la medición de indicadores cuantitativos específicos: tiempo de procesamiento de pedidos, tasa de errores en la carga de datos y nivel de actualización del stock.

1.6 Alcance y limitaciones

El sistema desarrollado abarca la gestión integral de productos, clientes y ventas, así como la automatización del proceso de recepción y registro de pedidos mediante la integración con la plataforma Telegram. El perfil de referencia utilizado para el diseño de los escenarios de evaluación corresponde a un comercio minorista de pequeña escala (por ejemplo, un almacén, kiosco o tienda de barrio). Sus características operativas —bajo volumen diario de pedidos, personal reducido y procesos de registro manuales— son representativas del segmento de PyMEs objeto de estudio.

El alcance del trabajo no incluye la integración con sistemas contables externos ni la gestión de múltiples sucursales, lo cual constituye una posible línea de trabajo futuro. Tampoco contempla la implementación de módulos de facturación electrónica ni de gestión de proveedores. La implementación piloto en un comercio real y su eventual comercialización tampoco forman parte del alcance de este trabajo final, y se plantean como líneas de continuidad (sección 7.2).

Se reconoce como limitación la dependencia del sistema respecto de la conectividad a internet y del correcto funcionamiento de los servicios externos utilizados (API de Telegram, servidor n8n). La limitación metodológica más relevante del presente trabajo es que la evaluación del sistema se realizó en un entorno de prueba controlado y no en un comercio real en operación. Esto se debe a que, a la fecha de este informe, el artefacto no se implementó comercialmente. En consecuencia, los resultados obtenidos poseen validez interna alta pero validez externa

acotada. Su generalización a condiciones reales de uso (variabilidad de clientes, picos de demanda, errores de conectividad en campo) requiere una validación de campo posterior, que se plantea como trabajo futuro prioritario (sección 7.2).

CAPÍTULO 2: Marco Teórico y Estado del Arte

2.1 Sistemas de gestión de inventario en PyMEs

Los sistemas de gestión de inventarios constituyen un elemento central dentro de la administración de operaciones, en tanto permiten controlar, registrar y optimizar el flujo de bienes dentro de una organización. De acuerdo con Heizer, Render y Munson (2023), una gestión eficiente del stock contribuye significativamente a la reducción de costos de almacenamiento y mejora el nivel de servicio al cliente mediante la disponibilidad oportuna de productos.

Desde una perspectiva sistémica, la gestión de inventarios no solo implica el control de cantidades disponibles, sino también la planificación de reposición, el análisis de rotación de productos y la minimización de pérdidas derivadas de obsolescencia o faltantes. Los sistemas informatizados superan las limitaciones de los métodos manuales al proporcionar herramientas de análisis en tiempo real y facilitar la toma de decisiones basada en datos.

En el contexto específico de las PyMEs, la adopción de sistemas de gestión de inventario enfrenta barreras particulares relacionadas con los costos de implementación, la resistencia al cambio organizacional y la falta de competencias técnicas internas. Moeuf et al. (2020) analizan estas barreras en un estudio sistemático sobre la adopción de tecnologías de la Industria 4.0 en pequeñas y medianas empresas manufactureras. Concluyen que las soluciones de bajo costo y fácil integración son las más adecuadas para este segmento. La integración de los sistemas de stock con módulos de ventas y gestión de clientes posibilita una visión integral del negocio, lo cual resulta fundamental en contextos de alta competitividad.

A los fines de este trabajo, se adopta una definición operativa de PyME basada en criterios cualitativos y cuantitativos combinados: organizaciones con gestión no profesionalizada de la propiedad y estructura jerárquica reducida. Según el criterio general de clasificación por tamaño de la OCDE (OECD, 2023), estas organizaciones se ubican en el segmento de micro y pequeñas empresas. El presente trabajo pone foco particular en comercios minoristas de atención al público con personal reducido, por ser el tipo de organización más representativo de los procesos manuales de gestión de pedidos que motivan este estudio.

2.2 Arquitectura API REST

El estilo arquitectónico REST (Representational State Transfer), introducido por Fielding (2000) en su tesis doctoral, se ha consolidado como uno de los enfoques más utilizados en el desarrollo de sistemas distribuidos. REST se fundamenta en un conjunto de restricciones que incluyen la separación entre cliente y servidor, la ausencia de estado en las comunicaciones (stateless), la utilización de una interfaz uniforme y la identificación de recursos mediante URIs (Uniform Resource Identifiers).

La adopción de este estilo arquitectónico permite desarrollar aplicaciones altamente escalables y flexibles, facilitando la interoperabilidad entre sistemas heterogéneos. El uso del protocolo

HTTP como mecanismo de comunicación y la representación de recursos en formatos estándar como JSON favorecen la integración con múltiples plataformas y dispositivos.

Richardson y Ruby (2007) destacan que la arquitectura REST simplifica el desarrollo y mejora la mantenibilidad del sistema al promover el desacoplamiento entre sus componentes. En años recientes, la proliferación de arquitecturas de microservicios ha reforzado la relevancia de REST como contrato de comunicación entre servicios independientes (Newman, 2019). Esta característica resulta especialmente valiosa en el contexto del presente trabajo, donde la API actúa como núcleo de integración entre el frontend, la base de datos y los servicios externos de automatización. Bogner, Kotstein y Pfaff (2023), en un experimento controlado con 105 participantes, aportan evidencia empírica de que el cumplimiento de las convenciones de diseño REST —como el uso semánticamente correcto de los verbos HTTP y la consistencia en la estructura de las respuestas— mejora significativamente la comprensión de la API por parte de quienes la consumen. Esta evidencia respalda las decisiones de diseño adoptadas para la API del presente trabajo, descritas en la sección 4.2. Un estudio empírico complementario, basado en 17 entrevistas realizadas en 11 empresas (Lercher et al., 2024), muestra que la evolución de las APIs de microservicios en la práctica enfrenta desafíos de comunicación entre equipos y de dependencia de los consumidores respecto de versiones antiguas, lo cual refuerza —desde la perspectiva del mantenimiento a largo plazo— la relevancia de las convenciones de diseño consistentes que Bogner et al. (2023) evalúan desde la comprensión inicial de la API. El versionado formal de la API excede el alcance del presente trabajo, pero esta evidencia resulta pertinente para su eventual evolución.

2.3 Separación frontend y backend

En el desarrollo de aplicaciones modernas, la separación entre frontend y backend constituye una práctica fundamental que permite organizar el sistema en capas claramente diferenciadas. El frontend se encarga de la interacción con el usuario: gestiona la presentación de la información y la experiencia de uso. El backend, en cambio, se ocupa de la lógica de negocio, el procesamiento de datos y la persistencia de la información.

Fowler (2002) señala que esta separación mejora la modularidad del sistema y facilita su escalabilidad y mantenimiento, ya que permite la evolución independiente de cada componente. En la actualidad, este patrón se implementa frecuentemente mediante frameworks de componentes del lado del cliente (React, Vue, Angular) que consumen APIs REST expuestas por el servidor. Para el presente trabajo, la elección de JavaScript como lenguaje del frontend responde a criterios de disponibilidad de herramientas, madurez del ecosistema y compatibilidad con los navegadores web actuales.

La implementación del frontend como aplicación de página única (Single-Page Application, SPA) tiene implicaciones directas sobre el diseño de la API REST del sistema. A diferencia de las aplicaciones web tradicionales de renderizado en servidor, una SPA delega toda la lógica de presentación al cliente y se comunica con el backend exclusivamente mediante llamadas HTTP asíncronas. Esto implica que la API debe exponer respuestas en formato JSON autocontenidas,

configurar los encabezados CORS (Cross-Origin Resource Sharing) cuando el frontend se sirve desde un dominio distinto del de la API (en este trabajo lo sirve el propio backend, por lo que el origen es el mismo), y diseñar sus endpoints de manera que cada respuesta incluya la información suficiente para que el cliente construya la vista sin requerir llamadas adicionales encadenadas. Este patrón de interacción se inscribe, en términos generales, en la arquitectura Modelo-Vista-Controlador (MVC), descrita junto con la separación en capas por Fowler (2002): el backend actúa como capa de Modelo y Controlador, gestionando la lógica de negocio y el acceso a datos, mientras que el frontend SPA asume el rol de Vista desacoplada. La comprensión de esta separación de responsabilidades es relevante para evaluar las decisiones de diseño de la API descritas en la sección 4.2.

2.4 Automatización de procesos de negocio

La automatización de procesos implica la utilización de tecnologías para ejecutar tareas repetitivas o estructuradas sin intervención humana directa. Davenport y Kirby (2016) sostienen que la automatización incrementa la eficiencia operativa, reduce la probabilidad de errores y libera recursos humanos para actividades de mayor valor agregado. En el ámbito empresarial, la automatización de procesos de negocio (Business Process Automation, BPA) abarca desde la ejecución de flujos de trabajo sencillos hasta la orquestación de sistemas complejos de múltiples componentes.

En el contexto de los sistemas de gestión comercial, la automatización resulta particularmente relevante en procesos como la recepción de pedidos, la actualización de inventarios y la generación de registros de ventas. van der Aalst (2016) proporciona un marco conceptual riguroso para el análisis y diseño de procesos de negocio. Señala que la automatización efectiva requiere una modelización precisa de los flujos de trabajo y una definición clara de los eventos que los desencadenan.

n8n es una plataforma de automatización de flujos de trabajo que permite definir pipelines mediante una interfaz visual basada en nodos (n8n, 2024). Su código es de acceso público bajo la Sustainable Use License, un modelo que la propia n8n denomina fair-code: el software es gratuito para uso interno y su código puede inspeccionarse y modificarse, pero restringe la redistribución comercial. Por eso mismo, n8n aclara explícitamente que no se considera de código abierto en el sentido estricto del término (n8n, 2022). Cada nodo encapsula una operación —llamadas HTTP, transformaciones de datos, integraciones con aplicaciones externas— y los flujos se activan mediante disparadores programados, webhooks o eventos manuales. Comparada con alternativas como Apache Airflow o Zapier, n8n ofrece una curva de aprendizaje más suave y una interfaz accesible para usuarios con distintos niveles de experiencia técnica. Esto resulta ventajoso en contextos donde operadores no especializados necesitan monitorear o modificar los flujos.

Trabajos recientes en el campo de la investigación aplicada confirman esta tendencia hacia la reducción de la barrera técnica de entrada. Sufi (2023) revisa sistemáticamente las plataformas low-code/no-code utilizadas en trabajos de investigación e identifica que este tipo de

herramientas —categoría dentro de la cual se enmarca n8n— permite a usuarios sin formación avanzada en programación construir flujos funcionales de automatización. Esto resulta particularmente pertinente para el segmento de PyMEs abordado en este trabajo, donde la disponibilidad de personal técnico especializado es limitada (sección 2.1). Desde una perspectiva más amplia de la automatización de procesos de negocio, Khantong y Sriboonlue (2026) sintetizan sistemáticamente 83 artículos arbitrados publicados entre 2015 y 2025 sobre la Automatización Robótica de Procesos (RPA) y proponen un marco integrado organizado en torno a la alineación estratégica, la ejecución operativa y la mejora continua. Si bien n8n no constituye una herramienta de RPA en sentido estricto —opera sobre integraciones y disparadores de eventos, no sobre la manipulación de interfaces de usuario existentes—, los autores identifican los mismos ejes de beneficios y barreras (bajo costo de adopción, gobernanza y mantenimiento) que motivan la elección de una herramienta low-code como capa de automatización en este trabajo.

2.5 Integración con plataformas de mensajería instantánea

Las plataformas de mensajería instantánea han evolucionado hasta convertirse en canales estratégicos de interacción comercial. Su adopción masiva por parte de los consumidores es un indicio de ello: Telegram alcanzó los 950 millones de usuarios activos mensuales en julio de 2024, cifra reportada por la compañía y difundida por medios especializados (Mehta, 2024). Esto convierte a estas plataformas en puntos de contacto naturales para la recepción de pedidos en comercios de pequeña escala.

Telegram ofrece una API robusta y bien documentada que permite la creación de bots capaces de interactuar automáticamente con los usuarios mediante comandos o mensajes de texto estructurado (Telegram, 2024). A diferencia de otras plataformas de mensajería, Telegram no impone restricciones comerciales sobre el uso de su API para bots, lo que facilita su integración en sistemas de gestión sin costos adicionales de licenciamiento.

La integración de chatbots en procesos comerciales ha sido objeto de estudio creciente en la literatura académica reciente. Adamopoulou y Moussiades (2020) realizan una revisión comprehensiva del estado del arte en chatbots, identificando sus aplicaciones en comercio electrónico, atención al cliente y automatización de tareas administrativas. Los autores señalan que la efectividad de estos sistemas depende en gran medida de la calidad de la integración con los sistemas backend, un desafío que el presente trabajo aborda mediante la utilización de n8n como capa de orquestación.

Corresponde precisar, sin embargo, que gran parte de la literatura sobre chatbots citada en esta sección aborda sistemas con capacidad de interpretación de lenguaje natural (NLP). El bot desarrollado en el presente trabajo (sección 4.4), en cambio, no incorpora procesamiento de lenguaje natural en ningún punto de su funcionamiento: la interacción con el cliente está guiada enteramente por botones en línea (inline keyboard) de Telegram, sin que el usuario escriba texto libre en ningún paso del proceso de toma de pedidos. La referencia a Adamopoulou y Moussiades (2020) se retoma con mayor precisión en la sección 2.7.2, en relación específica

con los chatbots basados en reglas —categoría en la que se inscribe el artefacto desarrollado—, y no con los sistemas basados en aprendizaje automático o NLP.

2.6 Bases de datos relacionales

Las bases de datos relacionales constituyen el núcleo de la persistencia de datos en sistemas de gestión. Su estructura basada en tablas, relaciones e integridad referencial garantiza la consistencia y seguridad de la información (Date, 2004). El modelo relacional, formalizado por E. F. Codd (1970), sigue siendo el paradigma dominante para sistemas transaccionales donde la consistencia de los datos es un requisito crítico.

MySQL, el sistema gestor de bases de datos relacionales utilizado en este trabajo, es una de las soluciones de código abierto más ampliamente adoptadas en el mundo. Tiene una presencia destacada en aplicaciones web y sistemas de gestión empresarial de pequeña y mediana escala. Su compatibilidad con el estándar SQL, su rendimiento en operaciones de lectura/escritura concurrentes y su integración nativa con frameworks de desarrollo web la convierten en una opción adecuada para el contexto del presente proyecto.

2.7 Estado del arte y vacío de conocimiento

La revisión de la literatura existente permite identificar un conjunto de trabajos que abordan, de manera parcial o combinada, los problemas que este trabajo pretende resolver.

2.7.1 Sistemas de gestión para PyMEs

Moeuf et al. (2020) realizan una revisión sistemática de la literatura sobre la adopción de tecnologías de la Industria 4.0 en PyMEs manufactureras. Identifican que las soluciones de gestión de inventario representan una de las áreas con mayor potencial de impacto, pero también con mayor resistencia a la adopción. Los autores señalan la necesidad de desarrollar herramientas de bajo costo, alta usabilidad y fácil integración con los sistemas existentes, lo cual constituye uno de los principios de diseño del artefacto desarrollado en este trabajo.

En el ámbito latinoamericano, estudios recientes han evidenciado que gran parte de las PyMEs del sector comercial continúa utilizando hojas de cálculo o registros manuales para la gestión de inventarios, incluso en contextos urbanos con acceso a tecnología (CEPAL, 2021). Esta realidad subraya la pertinencia de desarrollar soluciones tecnológicas adaptadas a las capacidades y restricciones de este segmento. Un análisis cuantitativo más reciente confirma esta tendencia: Parra-Sanchez y Talero-Sarmiento (2024), a partir de 448 documentos indexados en Scopus, documentan un crecimiento sostenido de la investigación sobre transformación digital en PyMEs. También documentan una intención creciente de estas organizaciones de adoptar tecnologías como la inteligencia artificial, el big data, la computación en la nube y el comercio electrónico, lo cual refuerza la pertinencia y oportunidad del artefacto desarrollado en este trabajo. Gao, Teh y Ho (2026) confirman esta tendencia mediante un análisis bibliométrico y de contenido de 115 estudios empíricos sobre transformación digital e innovación en PyMEs, publicados en 32 países y relevados en Scopus y Web of Science. Sus

resultados muestran una correlación positiva pero dependiente del contexto entre transformación digital e innovación, y los propios autores señalan que esta relación permanece insuficientemente explorada para el segmento de pequeñas y medianas empresas, lo cual refuerza el vacío de conocimiento identificado en la sección 2.7.3.

2.7.2 Automatización e integración de chatbots en sistemas de gestión

Adamopoulou y Moussiades (2020) categorizan los chatbots según su arquitectura (basados en reglas vs. basados en aprendizaje automático) y sus aplicaciones, destacando el comercio electrónico y la gestión de pedidos como áreas de alto impacto. Su análisis revela que los sistemas basados en reglas son particularmente adecuados para entornos donde los flujos de interacción son estructurados y predecibles, como la recepción de pedidos de un catálogo definido. En esta categoría se inscribe el bot de Telegram implementado en este trabajo, guiado enteramente por botones y sin componente de NLP (sección 2.5).

Fritsch (2024) revisa sistemáticamente 124 trabajos publicados entre 2019 y 2023 sobre los desafíos asociados a los chatbots. Identifica que persisten problemas clásicos —comprensión del lenguaje, entrenamiento y mantenimiento— junto con desafíos más recientes ligados a los modelos conversacionales basados en aprendizaje automático, como la interpretabilidad y los sesgos. El bot desarrollado en este trabajo, al ser un sistema basado en reglas y guiado enteramente por botones (sección 2.5), no incurre en la mayoría de estos desafíos: no requiere comprensión de lenguaje natural ni plantea problemas de interpretabilidad o sesgo. Esto tiene como contrapartida una interacción más restringida para el cliente. Li et al. (2026) analizan bibliométricamente 331 artículos arbitrados sobre chatbots en negocios y sistemas de información publicados entre 2018 y 2024. Confirman el crecimiento sostenido de esta línea de investigación y su interés creciente en cómo los usuarios interactúan con estos sistemas y sus consecuencias, lo cual sitúa la propuesta del presente trabajo dentro de un campo de investigación activo y en expansión.

Desde la perspectiva de la automatización de procesos, van der Aalst (2016) establece los fundamentos teóricos del análisis de procesos de negocio. Con ello, proporciona el marco conceptual necesario para justificar las decisiones de diseño de los flujos de automatización implementados con n8n en este trabajo. La utilización de modelos de proceso explícitos —aunque sean visuales y no formales— responde a la recomendación de documentar los flujos antes de automatizarlos.

2.7.3 Vacío de conocimiento identificado

La revisión de la literatura permite identificar con claridad el vacío de conocimiento que este trabajo pretende cubrir. Existen estudios sobre sistemas de gestión de inventario para PyMEs (Moeuf et al., 2020), trabajos sobre la integración de chatbots en comercio electrónico (Adamopoulou y Moussiades, 2020), propuestas de automatización de procesos de negocio (van der Aalst, 2016) y revisiones recientes sobre plataformas low-code/no-code (Sufi, 2023). Sin embargo, no se encontraron en la literatura revisada trabajos que integren de manera simultánea los tres componentes en una solución funcional orientada a pequeños comercios

de tipo presencial: (1) un sistema de gestión de stock basado en API REST, (2) un canal de recepción de pedidos mediante mensajería instantánea y (3) una capa de automatización de bajo costo basada en herramientas como n8n, de licenciamiento fair-code (sección 2.4).

Esta brecha justifica el desarrollo del artefacto tecnológico propuesto y define el aporte específico de este trabajo al campo de la Ingeniería de Software aplicada a contextos de PyMEs.

CAPÍTULO 3: Marco Metodológico

3.1 Tipo de investigación e hipótesis de trabajo

La presente investigación se enmarcó en un enfoque aplicado con orientación cuantitativa. Tuvo como finalidad el diseño, desarrollo e implementación de una solución tecnológica destinada a resolver una problemática concreta en el ámbito de la gestión comercial, y la evaluación de su impacto sobre variables operativas medibles.

Desde el punto de vista del diseño metodológico, se adoptó un diseño intrasujeto (within-subjects) con medición pretest-posttest en un único grupo. Este diseño permite comparar el desempeño de los procesos de gestión en dos condiciones: la modalidad tradicional, basada en procedimientos manuales o parcialmente digitalizados, y la modalidad automatizada, sustentada en el sistema desarrollado. Este diseño resulta pertinente en contextos donde no es posible la asignación aleatoria de unidades experimentales ni la conformación de un grupo de control independiente. Sí se dispone, en cambio, de escenarios comparables sobre los cuales medir el efecto de la intervención. En el marco de este trabajo, dicho diseño se instrumentó mediante un experimento controlado con escenarios de pedido estandarizados (sección 3.4), y no mediante la observación de un comercio real en funcionamiento. Esto respondió a la etapa de desarrollo en la que se encontraba el artefacto. La precisión terminológica de esta denominación —y su distinción respecto de un diseño cuasiexperimental en sentido estricto— se aborda en la sección 3.4.

La hipótesis de trabajo que guió el estudio es la siguiente:

H1: la implementación de un sistema integrado de gestión de stock basado en arquitectura API REST, con automatización de pedidos mediante n8n y Telegram, reducirá el tiempo promedio de procesamiento de pedidos en al menos un 50%, en comparación con la modalidad manual simulada, medido en un entorno de prueba controlado que reproduce las condiciones operativas típicas de un comercio minorista de pequeña escala.

H1 es falsable y operacionalizable mediante la variable definida en la sección 3.5; el diseño adoptado permitió contrastarla, y sus resultados se presentan en el Capítulo 5. TED y NAS se miden con el mismo diseño y se reportan de forma descriptiva, conforme a lo advertido en la sección 1.2, dado que un único operador, coautor del sistema, actuó en las dos condiciones.

3.2 Paradigma epistemológico

El estudio se inscribe en el paradigma post-positivista, que sostiene que el conocimiento científico puede aproximarse a la realidad mediante la observación sistemática, la medición objetiva y la contrastación empírica de resultados. Este paradigma reconoce, al mismo tiempo, las limitaciones inherentes a todo proceso de medición. En este marco, la validez de los hallazgos se fundamenta en la definición precisa de variables, la utilización de instrumentos de medición consistentes y la posibilidad de replicar el estudio en contextos similares.

La adopción de este paradigma resulta adecuada para investigaciones orientadas a evaluar el desempeño de sistemas tecnológicos mediante indicadores cuantificables, como es el caso del presente trabajo.

3.3 Enfoque de investigación: Design Science Research (DSR)

En términos de la clasificación propuesta por Hevner et al. (2004), el presente trabajo se enmarca en el enfoque de Ciencias del Diseño (Design Science Research, DSR). Este enfoque se centra en la creación y evaluación de artefactos tecnológicos como medio para generar conocimiento científico. El DSR se diferencia de la investigación comportamental tradicional en que su contribución primaria no es la descripción o explicación de fenómenos observables, sino el diseño de soluciones innovadoras a problemas relevantes, junto con la evaluación rigurosa de su efectividad.

El artefacto desarrollado en esta investigación es un sistema de gestión de stock basado en arquitectura API REST, integrado con herramientas de automatización de procesos y plataformas de mensajería. La contribución científica del trabajo radica en la evaluación empírica del impacto de dicho artefacto sobre la eficiencia operativa, la precisión de los datos y la optimización de los procesos comerciales en el entorno de estudio.

Hevner et al. (2004) establecen siete directrices para la conducción de investigaciones bajo el paradigma DSR. Las más relevantes para el presente trabajo son: (1) la producción de un artefacto tecnológico novedoso y funcional; (2) la evaluación rigurosa del artefacto mediante métodos apropiados; y (3) la comunicación del trabajo a audiencias tanto tecnológicas como de gestión. El diseño de la investigación presentado en este capítulo respondió a estas directrices. La demostración de utilidad mediante la aplicación del artefacto en un contexto real, en cambio, queda fuera del alcance de este trabajo y se plantea como línea de continuidad (sección 7.2).

3.4 Diseño de investigación

El diseño de investigación adoptado consistió en una medición pretest-posttest sobre un único grupo, ejecutada mediante un experimento controlado que simuló las condiciones operativas de un comercio minorista de pequeña escala. En este esquema, se evaluaron los indicadores operativos bajo la modalidad manual simulada (pretest) y se compararon con los resultados obtenidos tras la incorporación del sistema automatizado (posttest), sobre el mismo conjunto de escenarios de pedido. De ese modo, las diferencias observadas pudieron atribuirse de manera razonable a la introducción del sistema.

Corresponde precisar con mayor rigor conceptual la denominación de este diseño de investigación. Siguiendo la taxonomía clásica de Campbell y Stanley (1963), un diseño pretest-posttest de un único grupo, sin grupo de control independiente ni asignación aleatoria de unidades a condiciones, se clasifica en sentido estricto como preexperimental, y no como cuasiexperimental. Esta última categoría requiere, como mínimo, un grupo de comparación no equivalente. En el presente trabajo no fue posible constituir dicho grupo de comparación, dado

que un único operador, coautor del sistema, ejecutó el experimento sobre el mismo conjunto de escenarios en ambas condiciones. Esta precisión terminológica no invalida los resultados obtenidos, pero acota el alcance de las afirmaciones causales que pueden sostenerse a partir de ellos. El diseño intrasujeto reduce la variabilidad atribuible a diferencias entre sujetos u organizaciones, pero no descarta por sí solo otras fuentes de variación —como el efecto de aprendizaje discutido en la sección 3.9—, tal como sí lo haría un grupo de control independiente.

La unidad de análisis estuvo constituida por los procesos de gestión de pedidos y administración de stock. Estos se evaluaron sobre un conjunto de escenarios de prueba estandarizados que reproducen la operatoria típica de un comercio minorista de pequeña escala (la sección 3.5 detalla el tamaño y composición de dicho conjunto). En la condición pretest, los escenarios se resolvieron mediante el procedimiento manual (registro en planilla, sin el sistema desarrollado). En la condición posttest, en cambio, los mismos escenarios se resolvieron utilizando el sistema integrado, incluyendo el canal de Telegram y los flujos de n8n. Ambas condiciones se ejecutaron bajo las mismas reglas de asignación de escenarios para garantizar la comparabilidad de los resultados.

Para las condiciones pretest y posttest se diseñó un conjunto de 20 escenarios de pedido estandarizados por condición (precedidos por 5 escenarios de calentamiento no cronometrados, sección 3.9). Este conjunto replicó la variabilidad típica de un comercio minorista: pedidos de uno o múltiples productos, y casos límite como stock insuficiente o cantidad máxima permitida. El detalle completo de estos escenarios se presenta en la Tabla 3.1 y se documenta también en el Anexo A.

ID	Tipo	Categoría	Producto(s)	Cantidad	Resultado esperado
1	Simple 1 producto	Bebidas	Coca-Cola Regular	1	Confirmado
2	Simple 1 producto	Cigarrillos	Marlboro Box	1	Confirmado
3	Simple 1 producto	Golosinas	Chocolate Aireado Milka	1	Confirmado
4	Simple 1 producto	Perfumería e Higiene	Jabón Dove	1	Confirmado
5	Simple 1 producto	Golosinas	Doritos 250gr	1	Confirmado
6	Simple 1 producto	Bebidas	Coca-Cola Zero	1	Confirmado
7	Simple 1 producto	Cigarrillos	Phillips-Morris Convertible	1	Confirmado
8	Simple 1 producto	Golosinas	Bombón Bon o Bon	1	Confirmado
9	Simple 1 producto	Perfumería e Higiene	Pasta de dientes Colgate	1	Confirmado
10	Simple 1 producto	Bebidas	Pepsi Regular	1	Confirmado
11	Simple 1 producto	Bebidas	Sprite Regular	1	Confirmado
12	Simple 1 producto	Golosinas	Gomitas Mogul	1	Confirmado
13	Simple 1 producto	Bebidas	Cerveza Quilmes	1	Confirmado
14	Múltiple (3 productos)	Bebidas + Golosinas	Coca-Cola Regular + Fernet Branca + Doritos 250gr	1 c/u	Confirmado
15	Múltiple (2 productos)	Golosinas + Perfumería	Chocolate Aireado Milka + Pasta de dientes Colgate	1 c/u	Confirmado

16	Múltiple (2 productos)	Golosinas + Lácteos	Gomitas Mogul + Leche descremada LS	1 c/u	Confirmado
17	Error stock insuficiente	Lácteos	Queso crema La Serenísima	3	Cancelado (disp. 2 < 3)
18	Error stock insuficiente	Lácteos	Manteca La Serenísima	2	Cancelado (disp. 1 < 2)
19	Error stock insuficiente	Bebidas	Sprite Regular	4	Cancelado (disp. 2 < 4)
20	Límite cantidad máxima	Golosinas	Doritos 250gr	10	Confirmado (disp. 38 ≥ 10)

Tabla 3.1. Escenarios de pedido estandarizados por condición (n = 20), idénticos y ejecutados en el mismo orden en pretest y postest, precedidos por 5 escenarios de calentamiento no cronometrados (Anexo A, hoja Escenarios).

El tamaño de 20 observaciones por condición respondió a una restricción práctica de tiempo del equipo, dado que el experimento fue ejecutado íntegramente por los autores. Se optó por priorizar la verificación de la validez del contraste sobre la potencia estadística máxima, ya que la hipótesis H1 predice un efecto de magnitud grande (reducción del 50%), más fácil de detectar con muestras moderadas. La normalidad de las diferencias pareadas se verificó mediante la prueba de Shapiro-Wilk antes de aplicar la prueba t de Student para muestras pareadas, empleando su equivalente no paramétrico (prueba de Wilcoxon) en caso de incumplimiento. Con n=20 resultaba más probable recurrir a esta alternativa no paramétrica que con una muestra mayor, tal como efectivamente ocurrió (Capítulo 5).

Los 20 escenarios de pedido estandarizados por condición (Anexo A) no se seleccionaron mediante un procedimiento de muestreo aleatorio. Esto se debe a que no se dispuso de una población real de pedidos históricos del comercio de referencia a partir de la cual muestrear. Su construcción respondió, en cambio, a un criterio de diseño deliberado de los autores, orientado a maximizar la cobertura de la variabilidad operativa típica de un comercio minorista de pequeña escala. Se incluyeron así pedidos de uno y de múltiples productos, y casos límite (tres escenarios de stock insuficiente, IDs 17 a 19 del Anexo A, y un escenario de cantidad máxima permitida). La dimensión "cliente nuevo vs. recurrente", presente en una versión anterior del diseño de escenarios, se retiró de la versión final. El artefacto identifica al cliente únicamente por chat_id y por un campo de texto libre (sección 4.3), sin relación con una entidad de clientes, por lo que esa distinción no es representable en el sistema evaluado. Se reconoce como limitación que este criterio de construcción deliberada —al no ser aleatorio ni estar validado por un evaluador ajeno al equipo de desarrollo— puede introducir un sesgo de selección no cuantificable. Ese sesgo favorecería casos que los propios autores anticipan que el sistema resuelve correctamente. Adicionalmente, antes de los 20 escenarios de medición de cada condición se ejecutaron 5 escenarios de calentamiento, no cronometrados y excluidos de todo análisis. Con ellos se buscó que el operador superara la curva de aprendizaje básica del procedimiento (manual o automatizado) antes de iniciar el registro de datos; esta incorporación se retoma en la sección 3.9. Un único operador, coautor del sistema, ejecutó las dos condiciones. La grabación de pantalla la realizó el otro autor y los instantes de recepción y de

registro confirmado los leyó el mismo operador, con conocimiento de la condición de cada video.

3.5 Variables de estudio y operacionalización

Para la evaluación del sistema se definieron las siguientes variables dependientes, directamente relacionadas con la eficiencia operativa:

- **Tiempo de procesamiento de pedidos (TPP):** intervalo, expresado en segundos, entre la recepción del pedido por el canal correspondiente y su registro confirmado en el sistema. Se midió con un instrumento único en ambas condiciones: una grabación de pantalla con el reloj del sistema operativo visible (con segundos). Sobre esa grabación se identificó, para cada escenario, el instante de recepción del pedido y el instante en que quedó registrado y confirmado, leyendo el reloj al segundo en los dos casos. El pretest se ejecutó el 10/09/2026 y el posttest el 12/09/2026, con un intervalo de aproximadamente un día y 23 horas entre el cierre de uno y el inicio del otro (sección 3.9). Los horarios del pretest se habían cargado originalmente en formato a. m. por un valor por defecto del editor de planillas; como la corrida fue nocturna, lo que confirma la hora de guardado del video, se sumaron 12 horas a los dos extremos de cada intervalo, lo que no altera las diferencias que definen TPP. Antes de cada condición se resolvieron 5 escenarios de calentamiento no cronometrados, excluidos del análisis (sección 3.4).
- **Tasa de errores en la carga de datos (TED):** porcentaje de transacciones en las que se registró una incidencia de operación, es decir, un error que obligó a rehacer un paso del registro dentro de la misma transacción, calculado como la razón entre transacciones con incidencia y el total de transacciones registradas en el período de medición. Con excepción del escenario 14 del posttest, en el que el pedido se canceló y se volvió a simular (sección 6.2), ninguna de las incidencias registradas requirió una corrección posterior a la transacción. La naturaleza del error difiere entre condiciones: en el pretest es un error de registro en la planilla; en el posttest, como la interacción es íntegramente por botones y no se escribe texto libre, es un error de selección del operador. Por eso TED no compara exactamente la misma magnitud en ambas condiciones y se reporta solo de forma descriptiva. Se excluyen del cálculo de TED los rechazos por falta de stock: en esos casos el pedido capturó los datos correctamente y el sistema respondió según lo esperado. Por eso no constituyen un error de carga, sino una restricción de negocio correctamente aplicada. Estos rechazos se registran y describen por separado en la sección 5.1, pero no inciden sobre TED.
- **Nivel de actualización del stock (NAS):** proporción de productos auditados cuyo stock registrado —en el sistema, en el posttest, o en la planilla del operador, en el pretest— coincide con el stock esperado según el diseño de escenarios al cierre de la corrida experimental (pretest o posttest). El stock esperado se calculó como el stock inicial menos el consumo previsto por los escenarios confirmados (Anexo A, hoja Auditoria_Stock). Como el experimento es una simulación, sin mercadería ni un inventario físico independiente del registro, NAS verifica la consistencia interna del registro de stock y no su correspondencia con un inventario físico; esa limitación se

discute en la sección 6.1. La auditoría abarcó la totalidad de los 17 productos del catálogo por condición (no una muestra, ver sección 3.9).

La variable independiente fue la implementación del sistema integrado de gestión de stock con automatización de pedidos.

La operacionalización de estas variables se realizó mediante métricas cuantificables obtenidas a partir de una grabación de pantalla con el reloj del sistema visible (TPP), del registro de incidencias por escenario (TED) y de auditorías de cierre de stock (NAS), documentadas en el Anexo A.

3.6 Técnicas e instrumentos de recolección de datos

La recolección de datos se llevó a cabo mediante las siguientes técnicas:

- Pruebas funcionales del sistema, orientadas a verificar que cada componente respondiera correctamente a los casos de uso definidos.
- Pruebas de integración entre los distintos componentes del sistema (API REST, base de datos MySQL, frontend, Telegram y n8n), para verificar que la comunicación entre capas fuera correcta y robusta.
- Grabación de pantalla con el reloj del sistema operativo visible, leído al segundo, como instrumento único de medición del tiempo de procesamiento de pedidos en ambas condiciones (sección 3.5).
- Lectura, sobre esa grabación, del instante de recepción y del instante de registro confirmado de cada escenario, y anotación de las incidencias de operación que requirieron una corrección, en ambas condiciones.
- Auditorías de cierre de stock, que comparan el stock registrado con el esperado según el diseño de escenarios, para la medición del nivel de actualización del stock.

Estos instrumentos permitieron obtener datos objetivos y verificables para el análisis del desempeño del sistema, minimizando la subjetividad inherente a las mediciones basadas en estimaciones o percepciones.

Dado que la evaluación del sistema se realizó mediante un experimento controlado con escenarios simulados y no involucró a clientes reales del comercio, no se recolectaron datos personales de terceros durante esta etapa del trabajo. Los únicos participantes del experimento fueron autores del trabajo. En caso de que el sistema se implemente posteriormente en un comercio real, su operación deberá ajustarse a las disposiciones de la Ley N.º 25.326 de Protección de Datos Personales de la República Argentina. Esto aplica en particular a los datos de clientes gestionados a través de la plataforma de mensajería.

3.7 Herramientas y entorno tecnológico

El desarrollo del sistema se realizó utilizando el siguiente conjunto de herramientas tecnológicas, seleccionadas en función de su compatibilidad, escalabilidad, robustez y

disponibilidad como soluciones de código abierto, de bajo costo o de licenciamiento fair-code (sección 2.4):

- Backend: Java (versión 17 LTS) con el framework Spring Boot, por su madurez, amplia adopción en entornos empresariales y soporte nativo para la construcción de APIs REST mediante anotaciones.
- Frontend: JavaScript con HTML5 y CSS3, implementando una interfaz de usuario de página única (SPA) que consume la API REST expuesta por el backend.
- Base de datos: MySQL, por su compatibilidad con el estándar SQL, su rendimiento en operaciones transaccionales y su integración nativa con Spring Boot a través de JPA/Hibernate. En la corrida del experimento se utilizó MariaDB 10.4.28, el servidor de la distribución XAMPP, compatible con MySQL y accedido con su conector y su dialecto.
- Automatización: n8n (versión 2.35.7, la utilizada en la corrida del experimento, Anexo A), desplegado en un servidor local, utilizado para la orquestación de los flujos de trabajo que conectan el bot de Telegram con la API REST.
- Mensajería: API de Telegram (BotFather), para la creación y gestión del bot de recepción de pedidos.

3.8 Procedimiento de investigación

El proceso metodológico se estructuró en las siguientes etapas secuenciales:

1. Etapa 1 – Relevamiento y análisis de requerimientos: se identificaron las necesidades del sistema a partir de la experiencia de los autores con la operatoria de comercios minoristas de pequeña escala y de la revisión de la literatura (Capítulo 2). Esta etapa proporcionó los insumos para la definición de los requisitos funcionales y no funcionales del sistema.
2. Etapa 2 – Diseño de la arquitectura: se definieron los componentes del sistema, sus interfaces y el modelo de datos. Se priorizó la modularidad y el desacoplamiento entre capas para facilitar la evolución independiente de cada componente.
3. Etapa 3 – Desarrollo del backend y la API REST: se implementaron los endpoints para las operaciones CRUD sobre las entidades del dominio, con validación de datos y manejo de errores.
4. Etapa 4 – Desarrollo del frontend: se implementó la interfaz de usuario para la gestión de productos, clientes y ventas, consumiendo la API REST del backend.
5. Etapa 5 – Integración con Telegram y n8n: se configuró el bot de Telegram y se diseñaron los flujos de automatización en n8n para la recepción, procesamiento y registro automático de pedidos.
6. Etapa 6 – Pruebas funcionales y de integración: se verificó el correcto funcionamiento de cada componente de forma aislada y en conjunto, incluyendo escenarios de error y casos límite.
7. Etapa 7 – Evaluación del sistema: se midieron las variables operativas definidas en la sección 3.5, comparando los resultados obtenidos antes y después de la implementación del sistema.

3.9 Criterios de validez y confiabilidad

La validez interna del estudio se sustenta en la comparación controlada de las condiciones pre y post implementación, manteniendo constantes las demás variables del entorno operativo durante el período de medición. El carácter intrasujeto del diseño reduce la variabilidad atribuible a diferencias entre unidades de análisis, ya que el mismo conjunto de escenarios actúa como su propio punto de comparación. No obstante, el diseño adoptado no incorpora un grupo de control independiente ni asignación aleatoria, características que lo ubican, en sentido estricto, en la categoría de diseños preexperimentales (Campbell y Stanley, 1963; sección 3.4). Por eso, esta reducción de variabilidad intrasujeto no equivale a la robustez de inferencia causal que ofrecería un diseño con grupo de comparación no equivalente. Esta precisión acota el alcance de las afirmaciones causales sostenidas en el Capítulo 6, sin invalidar los hallazgos obtenidos.

La confiabilidad se sustenta en la replicabilidad de las pruebas realizadas: los procedimientos de medición están definidos de manera suficientemente precisa como para que otros investigadores puedan reproducirlos en contextos similares. El conjunto de datos crudos que respalda esta afirmación se documenta en el Anexo A. La utilización de métricas objetivas —tiempos de procesamiento leídos de una grabación de pantalla con reloj visible, incidencias de operación registradas escenario por escenario y auditorías de cierre de stock— reduce la posibilidad de sesgos en la recolección de datos.

Como amenaza a la validez interna, se reconoce el posible efecto de aprendizaje del operador, que podría contribuir de manera independiente tanto a la reducción de errores como a la reducción del tiempo de procesamiento de pedidos (TPP). Ambas condiciones se ejecutaron, en efecto, en un orden fijo (primero la modalidad manual, el 10/09/2026, luego la automatizada, el 12/09/2026) y sobre los mismos 20 escenarios, sin contrabalanceo ni más intervalo de lavado entre corridas que un día y 23 horas entre el cierre del pretest y el inicio del postest. Para mitigar parcialmente esta amenaza se incorporaron 5 escenarios de calentamiento antes de cada condición (sección 3.4), no cronometrados. Con ellos se buscó que el operador superara la curva de aprendizaje básica del procedimiento antes de que comenzara el registro de datos. Esta incorporación reduce, pero no elimina, el efecto de práctica residual sobre los 20 escenarios de medición —idénticos entre condiciones—, por lo que esta amenaza no puede descartarse por completo y se retoma como limitación en la sección 6.2.

Historial de corridas. El experimento no se ejecutó una sola vez. Una corrida original, anterior a las versiones previas de este informe, se descartó porque su resultado no podía reconstruirse a partir de datos verificables. La re-ejecución se realizó en septiembre de 2026: el pretest manual el 10/09; un primer intento de postest el 11/09, que se descartó porque reveló un defecto en la confirmación de pedidos (descuento doble de stock ante confirmaciones duplicadas casi simultáneas, sección 4.6); y el postest definitivo el 12/09. La decisión de descartar el intento del 11/09 se tomó después de observarlo y no respondió a una regla fijada de antemano. Sus tiempos no se usaron en el análisis principal, pero se conservan

(docs/analisis/rerun/intento1_postest_bug_confirmacion_doble/) y se informan en la Tabla 6.1, porque el postest definitivo fue, en la práctica, la segunda pasada completa de los mismos 20 escenarios con el circuito automatizado, lo que es relevante para la amenaza de práctica descrita en este apartado.

En cuanto a la validez externa, se reconoce como limitación central que el experimento se ejecutó en un entorno controlado y no en condiciones reales de operación comercial. Esto puede sobreestimar los beneficios observados respecto de un contexto con mayor variabilidad, interrupciones y comportamiento heterogéneo de clientes reales. Los resultados obtenidos deben interpretarse, por lo tanto, como una estimación del potencial del sistema bajo condiciones controladas, y no como una medición definitiva de su impacto en un comercio en funcionamiento. La validación en campo se plantea como trabajo futuro prioritario (Capítulo 7).

CAPÍTULO 4: Desarrollo del Sistema

4.1 Arquitectura del sistema

El sistema desarrollado se estructura bajo una arquitectura de tipo cliente-servidor, en la cual se establece una separación clara entre la capa de presentación (frontend) y la capa de lógica de negocio (backend). En este esquema, el frontend actúa como cliente y gestiona la interacción con el usuario. El backend, en cambio, expone una serie de servicios a través de una API REST que centraliza el procesamiento de datos y la ejecución de las operaciones del sistema.

La adopción de este modelo arquitectónico responde a principios de escalabilidad, modularidad y desacoplamiento, fundamentales en el desarrollo de sistemas modernos. El desacoplamiento entre cliente y servidor permite la evolución independiente de ambos componentes, facilitando futuras extensiones del sistema, como la incorporación de nuevas interfaces o servicios externos. Además, la arquitectura implementada favorece la mantenibilidad del sistema al organizar sus componentes en capas bien definidas. También permite la integración con servicios externos como Telegram y n8n, algo central para la automatización de procesos planteada en la investigación.

La capa de automatización, compuesta por los flujos de n8n y el bot de Telegram, opera de manera paralela al frontend tradicional. n8n procesa los pedidos recibidos a través del bot y los envía a la misma API REST que utiliza el frontend. Esto garantiza la consistencia de los datos y elimina la necesidad de mantener una lógica de negocio duplicada. El diagrama de arquitectura del sistema se presenta a continuación (Figura 4.1) y también se incluye en el Anexo D.

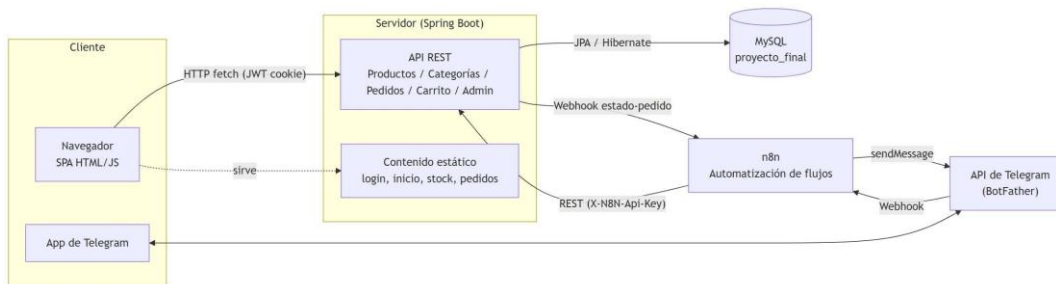


Figura 4.1. Diagrama de arquitectura del sistema: frontend SPA, backend Spring Boot, base de datos MySQL, y la capa de automatización (n8n + API de Telegram) que opera en paralelo consumiendo la misma API REST.

4.1.1 Implementación del frontend (SPA)

El frontend se implementó como una aplicación de página única (SPA) en JavaScript, HTML5 y CSS3, servida como contenido estático por el propio backend de Spring Boot. La interfaz se organiza en módulos funcionales independientes: autenticación (login), pantalla de inicio, gestión de productos, gestión de pedidos y control de stock. Cada módulo está compuesto por una vista HTML y un script JavaScript que consume la API REST mediante llamadas HTTP asíncronas (fetch) y renderiza dinámicamente la información recibida. El módulo de autenticación se implementó mediante JSON Web Tokens (JWT). Al iniciar sesión, el backend emite un token firmado que se almacena en una cookie con el atributo `HttpOnly` —inaccesible desde JavaScript, para mitigar ataques de tipo XSS— y que expira tras una hora de inactividad, renovándose automáticamente con cada solicitud autenticada (expiración deslizante). Un filtro de seguridad (Spring Security) exige una sesión válida para acceder a cualquier pantalla del panel de administración o a sus endpoints de gestión. Este filtro redirige al usuario no autenticado a la pantalla de inicio de sesión, tanto si intenta acceder directamente por URL como si su sesión expiró por inactividad. Los endpoints que consume la integración con Telegram y la consulta pública del catálogo usan mecanismos de autenticación distintos, detallados en la sección 4.2. Esta organización modular responde al objetivo específico OE3 (sección 1.5) y facilita que cada operador del comercio acceda únicamente a las funcionalidades relevantes para su tarea, manteniendo la separación de responsabilidades descrita en la sección 2.3. Las Figuras 4.1.1a a 4.1.1c muestran las pantallas de inicio de sesión, el panel de control y la gestión de productos; el resto de las capturas de la SPA y del bot se incluye en el Anexo E.

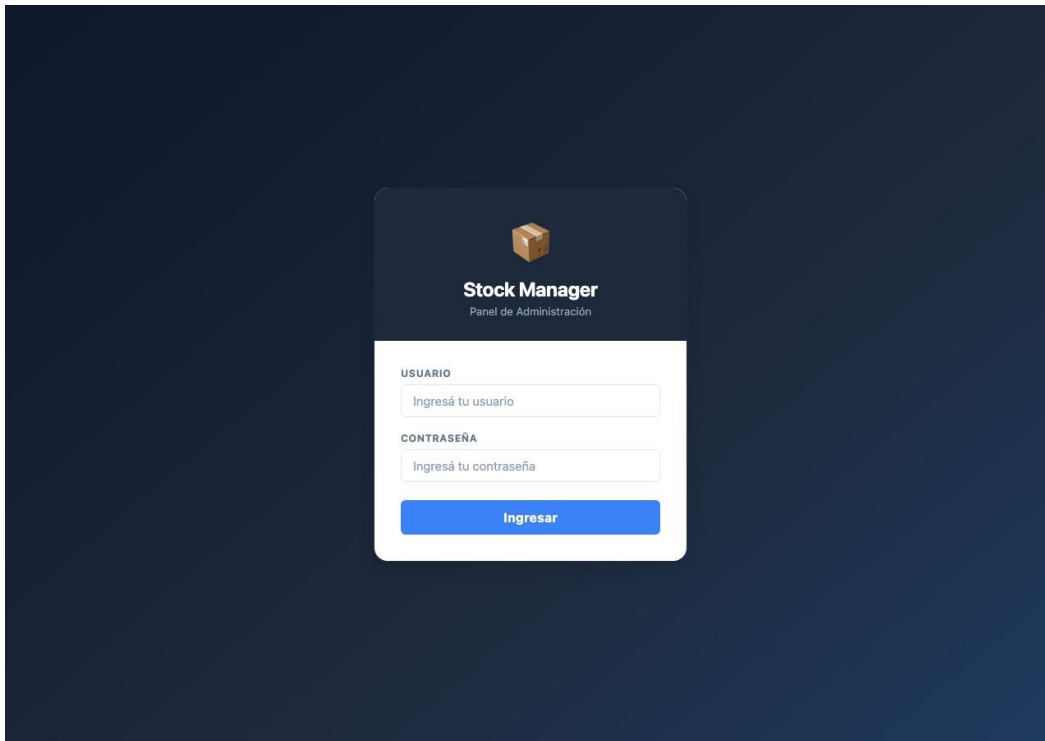


Figura 4.1.1a. Pantalla de login del panel de administración (autenticación JWT, sección 4.1.1).

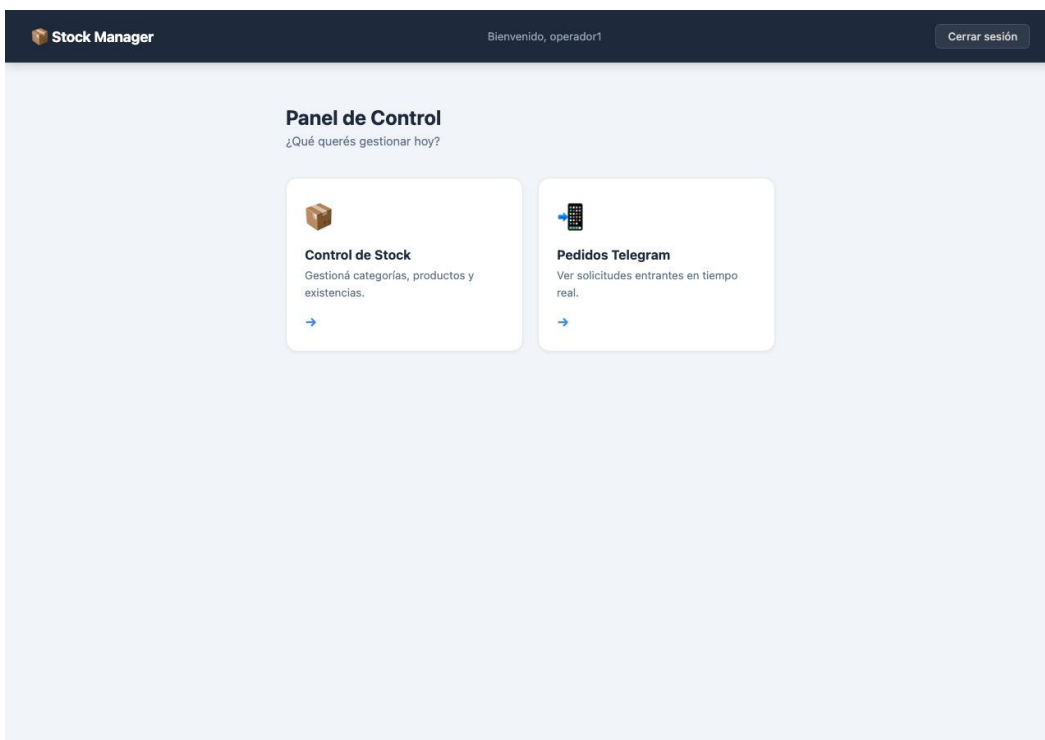


Figura 4.1.1b. Panel de control tras iniciar sesión, con acceso a Control de Stock y a Pedidos Telegram.

Categorías
Categoría: Bebidas
Stock Manager
Cerrar sesión

AGREGAR / EDITAR PRODUCTO

NOMBRE
DESCRIPCIÓN
PRECIO (\$)
STOCK

Guardar producto

BUSCAR PRODUCTO

PRODUCTOS

Mostrando 1-6 de 6 productos

NOMBRE	DESCRIPCIÓN	PRECIO	STOCK	ACCIONES
Cerveza Quilmes	Cerveza rubia 473 ml	\$1900	20	Editar Eliminar
Coca-Cola Regular	Gaseosa cola 500 ml	\$1300	23	Editar Eliminar
Coca-Cola Zero	Gaseosa cola sin azúcar 500 ml	\$1300	49	Editar Eliminar
Fernet Branca	Aperitivo 750 ml	\$10500	11	Editar Eliminar
Pepsi Regular	Gaseosa cola 500 ml	\$1200	11	Editar Eliminar
Sprite Regular	Gaseosa lima-limon 500 ml	\$1200	2	Editar Eliminar

Figura 4.1.1c. Módulo de gestión de productos (alta, edición y eliminación) dentro de una categoría del catálogo real del experimento.

4.2 Diseño de la API

El diseño de la API se fundamenta en los principios del estilo arquitectónico REST, garantizando una interfaz uniforme, el uso de operaciones estándar del protocolo HTTP y la representación de recursos mediante estructuras de datos en formato JSON.

Se definieron endpoints específicos para la gestión de las principales entidades del sistema: productos, categorías y pedidos (junto con sus ítems). Para cada entidad, la API expone las operaciones de creación (POST), consulta individual y colectiva (GET), actualización (PUT) y eliminación física (DELETE), siguiendo las convenciones semánticas del protocolo HTTP. El flujo de toma de pedidos se expone además mediante endpoints específicos de carrito (POST /carrito/{chatId}/item, POST /carrito/{chatId}/confirmar) para su integración con el bot de Telegram (sección 4.4). Este conjunto de operaciones cubre los requerimientos funcionales identificados durante el relevamiento inicial. El esquema de autenticación no es uniforme para toda la API. Las pantallas y endpoints de gestión del panel de administración (creación, edición y eliminación de productos, categorías y pedidos, y consulta de pedidos) exigen el JWT de sesión descrito en la sección 4.1.1. Los endpoints que consume la integración con el bot de Telegram vía n8n (POST /carrito/{chatId}/item, POST /carrito/{chatId}/confirmar, POST /pedidos/webhook), en cambio, se autentican con una clave compartida enviada en el encabezado X-N8N-API-Key, ya que esas llamadas no provienen de un administrador logueado. Las consultas de solo lectura del catálogo (GET /productos y GET /categorias, con sus variantes),

por último, son de acceso público y sin autenticación, para que el bot pueda armar los menús sin necesidad de credenciales.

Se implementaron mecanismos de validación de datos en la capa de la API: verificación de tipos, rangos y campos requeridos. También se implementó un sistema de manejo de errores que devuelve códigos de estado HTTP apropiados junto con mensajes descriptivos en formato JSON. Este enfoque mejora la confiabilidad del sistema y facilita el diagnóstico de problemas tanto en el desarrollo como en la operación del sistema.

4.3 Modelo de datos

El modelo de datos sigue el paradigma relacional y comprende seis entidades: productos, categorías, pedidos, items_pedido, usuarios y administradores (Anexo D). productos se vincula con categorías mediante clave foránea (`categoria_id`), permitiendo organizar el catálogo por rubros. Cada pedido agrupa uno o más registros de items_pedido —el detalle línea por línea de lo solicitado—, vinculados mediante `pedido_id` y referenciados a su vez a productos mediante `producto_id`. La entidad administradores es independiente del resto del modelo y corresponde a las cuentas de los operadores del comercio que acceden a la interfaz web (sección 4.1.1).

Cada ítem de pedido almacena, además de la referencia al producto (`producto_id`), una copia de su nombre y precio al momento de la compra (`nombre`, `precio`, `subtotal`). Esta duplicación es intencional: desacopla el historial de pedidos de cambios posteriores en el catálogo (por ejemplo, una actualización de precio no debe alterar pedidos ya registrados). Es un patrón habitual en sistemas de ventas, conocido como snapshot de línea de pedido, y constituye una excepción deliberada a la normalización estricta. La entidad usuarios está diseñada para representar a los clientes del comercio, con una relación de uno a muchos hacia pedidos (un cliente puede realizar múltiples pedidos, cada pedido pertenece a un único cliente). Esta relación corresponde al diseño planificado del modelo y se encuentra pendiente de implementación al momento de este informe. Actualmente, pedidos identifica al cliente únicamente mediante `chat_id` y el campo de texto libre `cliente_nombre`, sin clave foránea hacia usuarios, tal como puede verificarse en el diagrama entidad-relación presentado a continuación (Figura 4.2) y en el Anexo D.

La decisión de utilizar MySQL como sistema gestor responde a su compatibilidad con el estándar SQL y a su integración nativa con Spring Boot a través de JPA (Java Persistence API) e Hibernate. Esto simplifica las operaciones de mapeo objeto-relacional y reduce la cantidad de código repetitivo en la capa de persistencia.

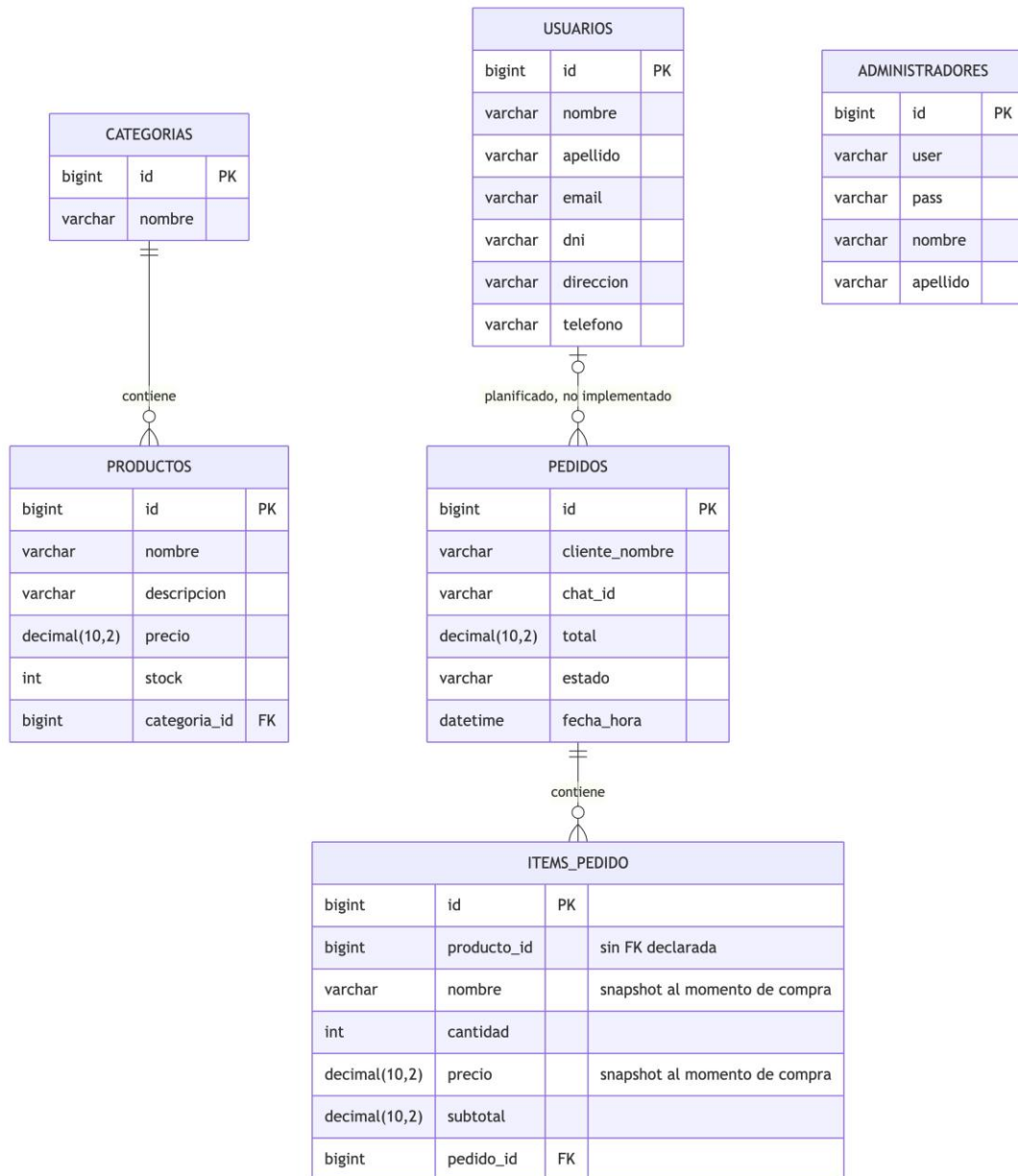


Figura 4.2. Diagrama entidad-relación de la base de datos, correspondiente a las seis entidades descritas en esta sección. La relación usuarios–pedidos se muestra en línea punteada por estar planificada pero no implementada al momento de este informe.

El mismo modelo se presenta a continuación desde una perspectiva de objetos, como diagrama de clases UML (Figura 4.3). Este diagrama hace explícitas la composición entre Pedido e ItemPedido (los ítems se persisten y eliminan junto con el pedido que los contiene, sin existencia propia) y la referencia sin clave foránea declarada entre ItemPedido y Producto, discutida más arriba.

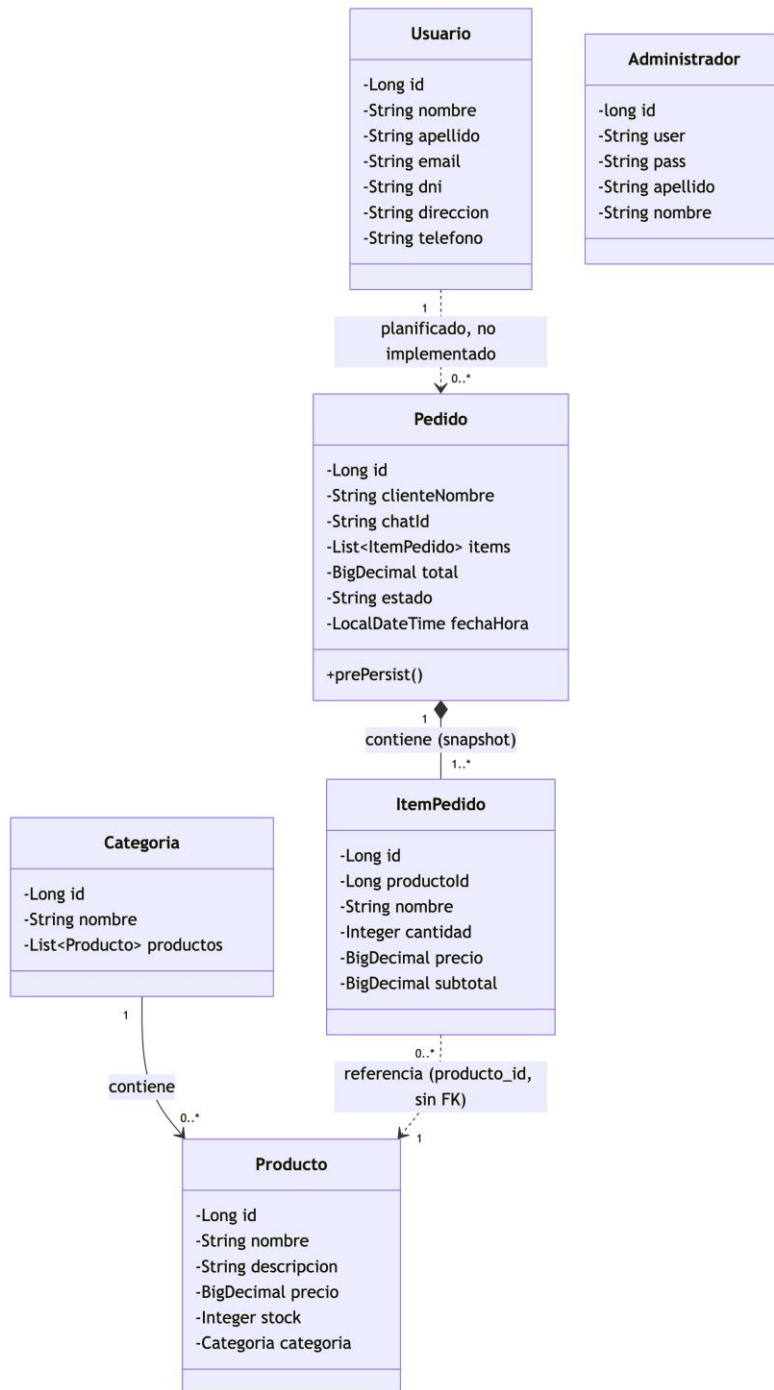


Figura 4.3. Diagrama de clases UML del modelo de dominio, derivado directamente de las entidades JPA del código fuente (paquete model). Los rombos representan composición; las flechas punteadas, dependencias sin clave foránea o relaciones planificadas y no implementadas.

4.4 Integración con Telegram y automatización con n8n

Uno de los aportes más significativos del sistema desarrollado radica en la integración con plataformas de mensajería instantánea y la utilización de herramientas de automatización para

la gestión de flujos de trabajo. Esta integración permite que los usuarios —clientes del comercio— puedan enviar pedidos a través de Telegram sin necesidad de interactuar directamente con la interfaz web del sistema.

La interacción del cliente con el bot es enteramente guiada mediante botones en línea (inline keyboard) de Telegram, sin que el cliente escriba texto libre en ningún momento del proceso. El flujo, implementado en n8n, distingue entre mensajes de texto y `callback_query` (pulsaciones de botón) mediante un nodo condicional. Cualquier mensaje de texto —incluyendo el comando `/start`— dispara una consulta `GET /categorias` a la API REST y presenta el menú de categorías disponibles. A partir de allí, el cliente navega exclusivamente por botones: selecciona una categoría, luego un producto de esa categoría y finalmente una cantidad entre 1 y 10 unidades mediante botones numerados. Este límite de diez unidades, impuesto originalmente solo por la construcción del teclado en n8n, se reforzó en una iteración posterior con una validación equivalente del lado del servidor: el endpoint `POST /carrito/{chatId}/item` rechaza con código HTTP 400 cualquier cantidad fuera del rango 1-10. De ese modo, el servidor ya no confía exclusivamente en el límite impuesto por el cliente. Cada selección de cantidad agrega el ítem correspondiente al carrito del cliente mediante una llamada `POST /carrito/{chatId}/item`.

Al finalizar la selección, el cliente confirma el pedido mediante un botón dedicado, lo que dispara una llamada `POST /carrito/{chatId}/confirmar`. El backend valida la disponibilidad de stock al momento de crear el pedido: si el stock es insuficiente, el pedido se registra con estado CANCELADO, pero la API responde igualmente con código HTTP 200. Por ello, el flujo de n8n distingue este caso inspeccionando el campo estado del cuerpo de la respuesta —no el código de estado HTTP— para notificar al cliente que no hay stock suficiente y vaciar su carrito. Si el pedido se procesa correctamente, el flujo envía al cliente una confirmación con el detalle y el total. Posteriormente, cuando un operador del comercio confirma o cancela el pedido desde la interfaz web, el backend notifica ese cambio de estado a un webhook expuesto por n8n, que reenvía el mensaje correspondiente al cliente por Telegram. Al confirmar el pedido, el backend descuenta el stock de cada ítem mediante una operación atómica en la base de datos; al cancelarlo, no modifica el stock (Figura 4.4c). A diferencia de lo que podría sugerir un diseño basado en texto libre, el sistema actual no incorpora ninguna lógica de interpretación de lenguaje natural: toda la información del pedido proviene de callbacks estructurados generados por los propios botones. Esto reduce la ambigüedad del proceso de toma de pedidos, a costa de restringir al cliente a las opciones predefinidas en el catálogo. El flujo completo de n8n, con la totalidad de sus nodos, se incluye exportado como Anexo C. Los pedidos recibidos por Telegram se listan en el panel de administración con filtros de búsqueda (Figura 4.4a); el flujo completo, desde la selección del cliente hasta la notificación del cambio de estado, se representa como diagrama de secuencia (Figura 4.4b) y como diagrama de proceso por carriles (Figura 4.4c).

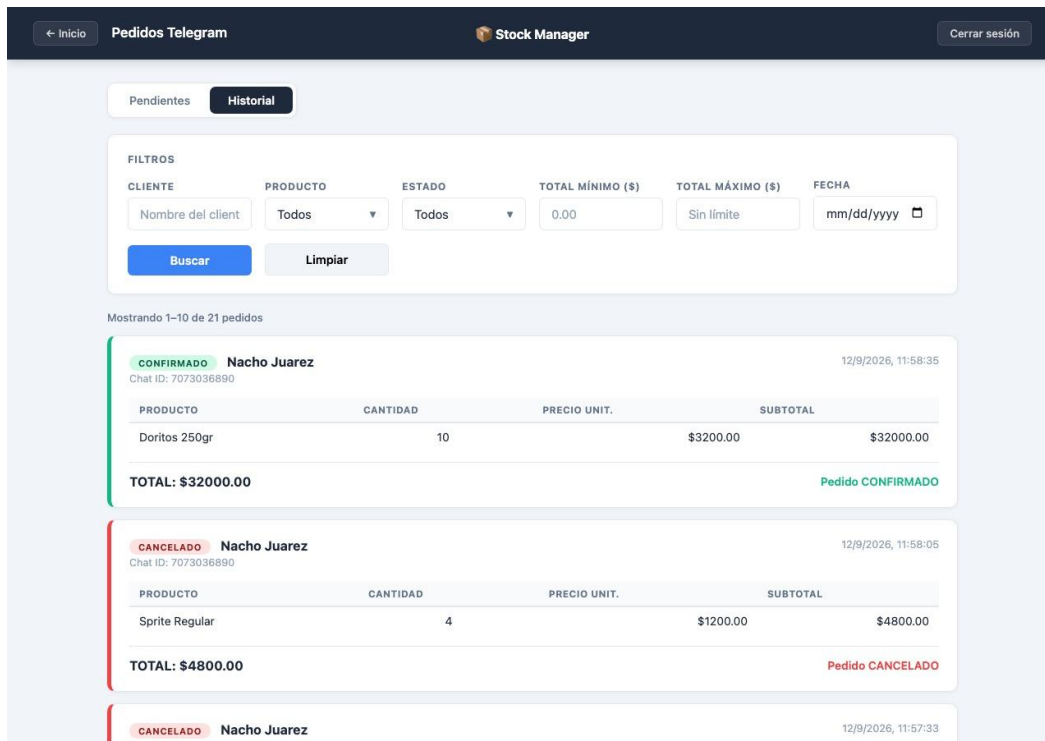


Figura 4.4a. Historial de pedidos recibidos por Telegram en el panel de administración, con filtros de búsqueda (cliente, producto, estado, rango de total y fecha). Datos reales de la re-ejecución del experimento de septiembre de 2026 (sección 3.9), pasada suplementaria del 12/09.

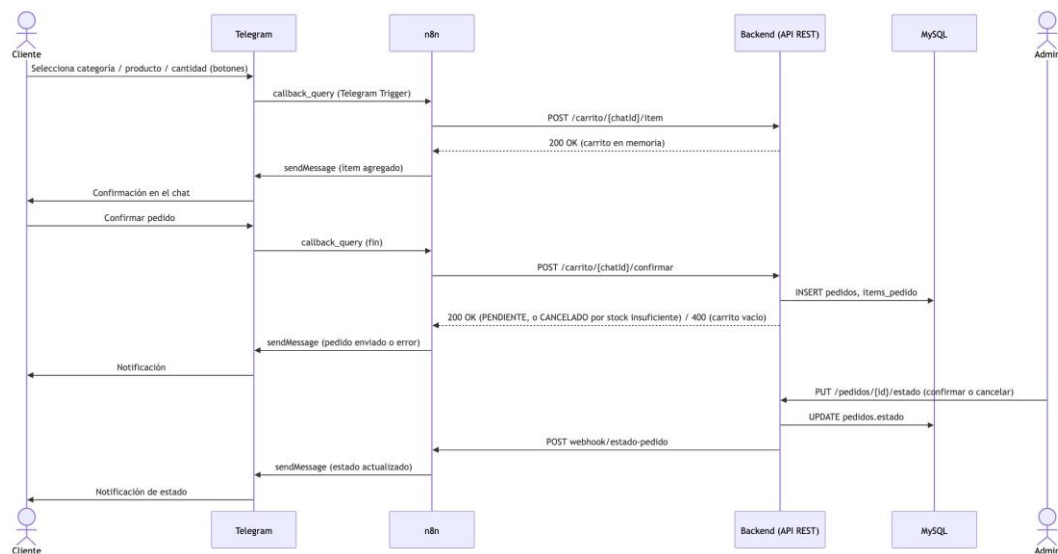


Figura 4.4b. Diagrama de secuencia del flujo completo: selección y confirmación del pedido por el cliente vía Telegram/n8n, y notificación del cambio de estado desde el panel de administración de vuelta al cliente.

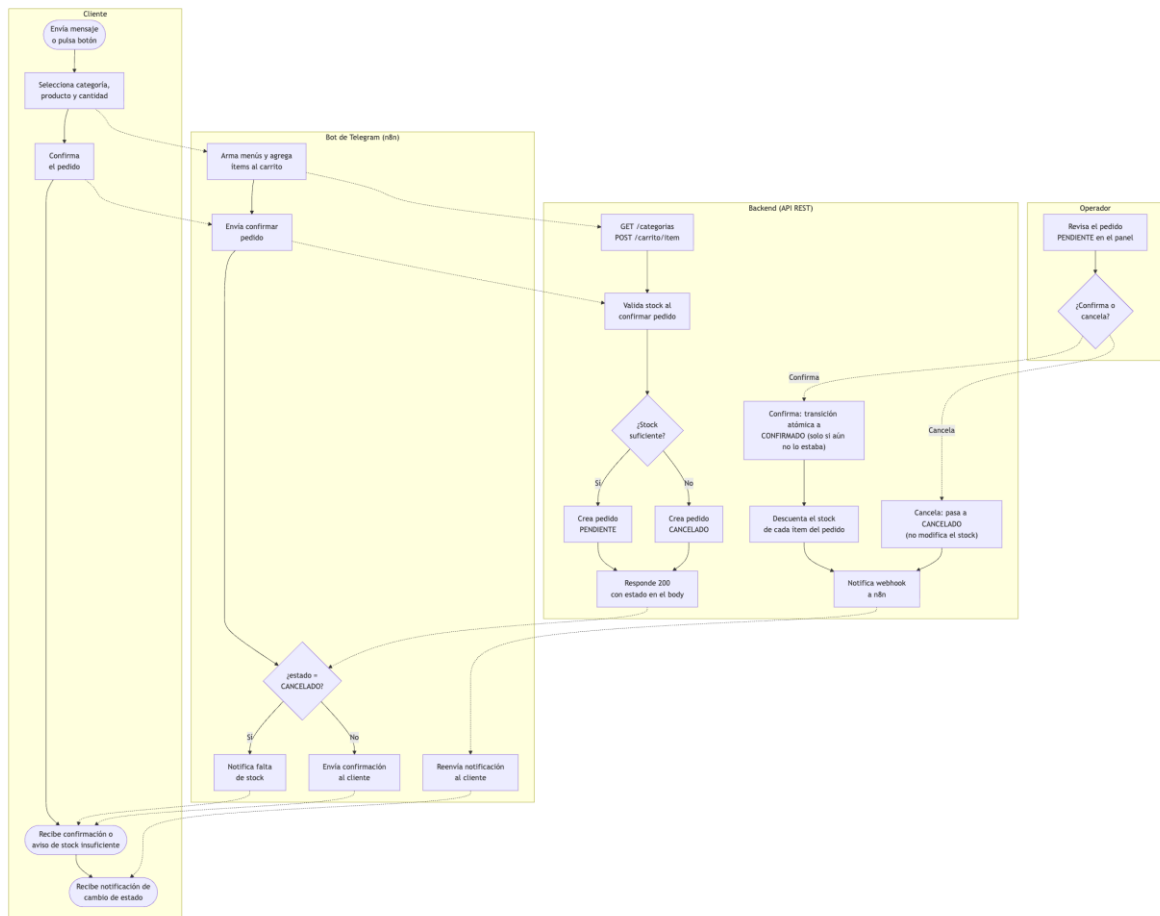


Figura 4.4c. Diagrama de proceso por carriles (un carril por actor) del flujo de toma de pedidos por Telegram descrito en esta sección: Cliente, Bot de Telegram (n8n), Backend (API REST) y Operador. Complementa, a nivel de proceso de negocio, el diagrama de secuencia técnico de la Figura 4.4b, e incluye el descuento de stock, que ocurre cuando el operador confirma el pedido y no cuando el bot lo recibe. Fuente: docs/diagramas/diagrama_bpmn_pedido.mmd (repositorio de código, Anexo F).

4.5 Pruebas del sistema y validación del artefacto

Con el objetivo de garantizar el correcto funcionamiento del sistema y validar su desempeño, se realizaron distintas instancias de prueba abarcando múltiples niveles de análisis, conforme al enfoque de evaluación del artefacto propio del paradigma DSR (Hevner et al., 2004).

Durante el desarrollo se realizaron verificaciones funcionales manuales orientadas a comprobar el comportamiento de los componentes de la API —particularmente la lógica de validación de datos y el manejo de errores en cada endpoint. Estas verificaciones se llevaron a cabo consumiendo la propia API desde el frontend de la aplicación y, en el caso de los endpoints vinculados a la toma de pedidos, mediante interacción directa con el bot de Telegram. En cada caso se observaron las respuestas devueltas y el comportamiento resultante en la base de datos. No se utilizó una herramienta dedicada de prueba de peticiones HTTP (como Postman) ni una especificación OpenAPI. Las pruebas de los endpoints CRUD (productos, categorías,

pedidos) se ejecutaron, en cambio, a través de las mismas pantallas de la SPA que los consume en producción: esto permitió verificar el comportamiento end-to-end del sistema tal como lo experimentaría un operador real, aunque no permitió aislar cada endpoint de forma independiente del resto de la interfaz. Las capturas incluidas en el Anexo E documentan estas pruebas sobre la SPA y sobre el bot.

Asimismo, se verificó de forma manual la correcta interacción entre los distintos módulos del sistema: la comunicación entre el frontend y la API, la comunicación entre la API y la base de datos MySQL, y la comunicación entre n8n, el bot de Telegram y la API REST. Adicionalmente, se incorporó una suite de pruebas de integración automatizadas (JUnit 5 y Spring Boot Test, con MockMvc) que cubre específicamente los endpoints de carrito y confirmación de pedido que sostienen los resultados del Capítulo 5. Esta suite prueba: la validación de cantidad máxima del lado del servidor; el escenario de carrito vacío; la confirmación de pedidos con stock suficiente e insuficiente; el descuento de stock al pasar un pedido a estado CONFIRMADO; la ausencia de descuento duplicado ante una confirmación repetida del mismo pedido; la acumulación de cantidades de un mismo producto en el carrito por encima del máximo de 10 unidades por interacción y su validación contra el stock real al confirmar; el filtrado del historial de pedidos por rango de fecha; y el control de acceso mediante autenticación JWT sobre los endpoints administrativos. La suite se ejecuta de manera repetible mediante Maven y se documenta en el repositorio de código (Anexo F).

Adicionalmente, se ejecutaron manualmente escenarios de extremo a extremo que simulan casos reales de uso: la recepción de un pedido a través de Telegram, su procesamiento automático por n8n, el registro correspondiente en la base de datos y la actualización del stock disponible. Estos escenarios cubrieron tanto casos de éxito como los casos de error más frecuentes identificados durante el relevamiento de requerimientos (producto inexistente, stock insuficiente, formato de mensaje inválido). Estos mismos escenarios constituyen la base para los escenarios de prueba estandarizados del experimento controlado descrito en la sección 3.4 y documentados en el Anexo A.

Los resultados de las pruebas permitieron identificar y corregir inconsistencias en la validación de datos, optimizar la estructura de algunos endpoints y ajustar los flujos de n8n para mejorar su robustez frente a mensajes malformados. Esta etapa de pruebas iterativas es coherente con el enfoque DSR, que concibe el desarrollo y la evaluación del artefacto como un proceso cíclico orientado a la mejora continua.

4.6 Evolución del artefacto durante y después del experimento

Esta sección reúne las modificaciones que recibió el artefacto en torno al experimento descrito en el Capítulo 5, en orden cronológico y con el commit del repositorio que respalda cada una (Anexo F). Se trata de trabajo ya realizado; lo que queda pendiente se detalla en la sección 7.2. La suite de pruebas automatizadas que protege estos cambios se describe en la sección 4.5. Los resultados del Capítulo 5 corresponden al árbol de trabajo local de la corrida del 10 y el 12/09/2026, cuyo estado se consolidó después en el commit 97ce027. Los commits posteriores (95af526, 4c57af8 y 15625b8) modifican los tipos numéricos y de fecha, el filtro del historial de

pedidos, la configuración de despliegue y la seguridad, y no alteran la secuencia de carrito, confirmación y notificación que se midió; el cálculo de subtotal y total pasó a BigDecimal sin cambiar su resultado.

El 11/08/2026, con anterioridad a la corrida del 10 y el 12/09/2026, se resolvieron dos tareas técnicas puntuales: la corrección de la excepción no controlada que exponía el escenario de carrito vacío, y la incorporación de una validación de cantidad máxima del lado del servidor. Esta última cantidad estaba antes impuesta únicamente por la construcción de los botones en n8n (sección 4.4). Para el primer caso, se agregó manejo explícito de la excepción (respuesta HTTP 400 con mensaje descriptivo; commit 1d95abf del repositorio, Anexo F). Para el segundo, una validación de rango (1 a 10 unidades) en el endpoint POST /carrito/{chatId}/item (commit 237ed83 del repositorio, Anexo F).

Durante la re-ejecución del experimento y en la revisión de código posterior se identificaron dos observaciones sobre el flujo de carrito y confirmación de pedidos, que se investigaron y resolvieron. La primera era una condición de carrera en la confirmación de pedidos: la comprobación en memoria del estado antes de descontar stock permitía que dos confirmaciones casi simultáneas del mismo pedido (por ejemplo, un doble clic) leyeran ambas el estado no confirmado antes de que la primera terminara de escribir, descontando el stock dos veces. Se corrigió reemplazando esa comprobación por una transición atómica a nivel de base de datos: una sentencia UPDATE condicionada a que el estado no sea ya CONFIRMADO, el mismo patrón ya usado para el descuento de stock. La corrección se validó con diez confirmaciones simultáneas reales sobre el mismo pedido y con el test de regresión confirmarEstadoDosVecesNoDescuentaStockDosVeces (sección 4.5; commit 97ce027 del repositorio, Anexo F). La segunda cuestionaba si la cantidad acumulada de un mismo producto al agregarlo dos veces al carrito debía limitarse al máximo de 10 unidades por interacción impuesto por el menú de botones de Telegram (sección 4.4). Se determinó que ese límite es exclusivo de la interfaz de botones y no una regla de negocio, por lo que la cantidad acumulada puede superar las 10 unidades sin inconveniente, siempre que la disponibilidad se valide contra el stock real al confirmar el pedido. Este comportamiento ya lo implementaba correctamente el sistema, y quedó cubierto con dos tests de regresión adicionales (sección 4.5).

También se identificó, como riesgo de seguridad, que la configuración de desarrollo versionaba credenciales sensibles: un usuario de MySQL sin contraseña y valores por defecto para el secreto de firma JWT y la clave compartida de la integración con n8n. Estos valores permitían levantar el sistema sin definir ningún secreto real. Se separó la configuración en un perfil "dev" —activo por defecto, sin cambios en el flujo de trabajo local ni en la suite de pruebas— con esos valores de conveniencia. El archivo de configuración base, en cambio, no tiene valores por defecto para las credenciales de base de datos, el secreto JWT y la clave de n8n: cualquier otro perfil de despliegue debe definir las por variable de entorno, o el arranque falla con un error explícito (commit 97ce027 del repositorio, Anexo F). Se agregó también la posibilidad de marcar la cookie de sesión JWT como Secure (solo HTTPS) mediante una variable de entorno, deshabilitada por defecto para no romper el desarrollo local sobre HTTP. Se agregó, por último, un perfil "prod" (application-prod.properties) que sobrescribe spring.jpa.hibernate.ddl-auto

de “update”, conveniente mientras el modelo de datos todavía cambiaba en desarrollo, a “validate”: en un despliegue real, Hibernate compara el esquema existente contra las entidades y falla el arranque si no coinciden, en lugar de crear o alterar tablas automáticamente (commit 4c57af8 del repositorio, Anexo F). En una revisión posterior se cerraron dos observaciones de seguridad que quedaban abiertas: se quitó el origen abierto (@CrossOrigin con origen “*”) de los controladores de pedidos y de carrito, dado que el frontend se sirve desde el propio backend y n8n llama a la API de servidor a servidor, y server.error.include-message quedó en “never” en el archivo base, de modo que solo el perfil “dev” expone el mensaje de las excepciones no controladas (commit 15625b8 del repositorio, Anexo F). La protección contra CSRF sigue deshabilitada, como se declara en el Anexo F.

Además, se migraron los tipos de datos que el modelo declaraba como Double (precio, subtotal y total), String (fecha y hora del pedido) y long (teléfono). Precio, subtotal y total pasaron a BigDecimal (DECIMAL(10,2) en la base de datos), la fecha y hora del pedido a LocalDateTime (DATETIME) y el teléfono a String (VARCHAR), tanto en las entidades JPA como en el esquema documentado (docs/analisis/schema.sql). La consulta del historial de pedidos, que filtraba por fecha comparando cadenas, se reescribió como un rango de día [00:00, 24:00) y quedó cubierta por un test propio (sección 4.5; commit 95af526 del repositorio, Anexo F). Además, Usuario.telefono se anotó con longitud 20 y sin valores nulos, de modo que el esquema que genera Hibernate coincide con el documentado (commit 15625b8 del repositorio, Anexo F). Las Figuras 4.2 y 4.3 reflejan estos tipos.

CAPÍTULO 5: Resultados

5.1 Resultados del experimento controlado

Esta sección presenta los resultados obtenidos tras la ejecución del protocolo experimental descrito en la sección 3.4, una vez finalizadas las condiciones pretest y posttest sobre los 20 escenarios de pedido estandarizados por condición. Los datos crudos —horarios de inicio y fin registrados para cada escenario, la ocurrencia de errores de carga y los resultados de las auditorías de stock— se documentan íntegramente en la planilla incluida como Anexo A. A partir de esos datos se calcularon los estadísticos que se presentan en la Tabla 5.1.

Variable	Pretest (media ± DE)	Posttest (media ± DE)	Diferencia observada	Valor p	Tamaño del efecto / IC95%	¿Cumple umbral H1?
TPP – Tiempo de procesamiento de pedidos (s)	39,30 ± 19,51	25,60 ± 10,52	13,70 s (-34,9%)	p = $9,54 \times 10^{-6}$ (Wilcoxon)	r = 0,60 (grande, N=40); IC95% bootstrap mediana de dif.: [8; 16] s	No (umbral 50%)
TED – Tasa de errores en la carga de datos (%)	11,8%	11,8%	0,0 p.p. (0%)	p = 1,000 (McNemar exacto)	No aplica	No aplica
NAS – Nivel de actualización del stock (%)	100,0%	100,0%	0,0 p.p. (0%)	No calculado (n=17 por condición; sin variación entre condiciones)	No aplica	No aplica (TED y NAS son indicadores descriptivos, sin umbral fijado a priori)

De los 20 escenarios diseñados (sección 3.4), la Tabla 5.1 presenta indicadores calculados sobre denominadores distintos según la variable; esto se explicita a continuación para evitar ambigüedades. TPP se calculó sobre los 20 escenarios completos, ya que en todos los casos —incluidos los tres de stock insuficiente— existe un tiempo de procesamiento medible entre la recepción del pedido y su registro. TED se calculó sobre 17 de los 20 escenarios, excluyendo los tres escenarios de stock insuficiente (IDs 17 a 19 del Anexo A). En ambas condiciones el sistema los resuelve mediante un rechazo correcto por falta de stock, por lo que no constituyen un error de carga de datos ni aportarían variabilidad relevante a esta variable. NAS, por su parte, no se midió sobre los escenarios de pedido sino de forma independiente, mediante una auditoría de stock sobre la totalidad de los 17 productos del catálogo por condición (sección 3.5). El escenario de carrito vacío (sección 6.2) no forma parte de los 20 escenarios cuantitativos: se excluyó por no tener equivalente en la modalidad manual y se verificó de forma cualitativa, por fuera del análisis estadístico.

Tabla 5.1. Comparación de resultados entre la condición pretest y posttest del experimento controlado (n = 20 escenarios por condición, sección 3.4). El tamaño del efecto de TPP se calculó

a partir de los mismos datos pareados del Anexo A, mediante el estadístico $r = |z|/\sqrt{N}$ asociado a la prueba de Wilcoxon (Rosenthal, 1991), que se informa en valor absoluto porque el signo de z depende de la convención de la diferencia y no cambia la magnitud, interpretado según los umbrales convencionales de Cohen (1988). El denominador N corresponde a las 40 observaciones totales (pretest + posttest), no a los 20 pares evaluados. El valor p informado es el valor exacto de la prueba de Wilcoxon (principal); la aproximación normal ($z = -3,79$) da un valor $p = 0,000149$, del mismo orden de magnitud. El intervalo de confianza bootstrap percentil corresponde a la mediana de las diferencias pareadas, no a la diferencia de las medianas de cada condición por separado, que es una cifra distinta (32,00 s vs. 21,50 s = 10,50 s). Se calculó con 10.000 remuestreos y semilla fija (`random_state = 42`) para garantizar la reproducibilidad. La mediana observada de las diferencias pareadas fue de 11,50 s. Su cociente sobre la mediana del pretest (35,9%) y la reducción de las medianas de cada condición (32,8%) se informan con su intervalo de confianza en la sección 5.2. Para TED se aplicó la prueba exacta de McNemar ($p = 1,000$), dado que los errores se registraron en escenarios distintos entre condiciones (2 pares discordantes en cada sentido). No corresponde, por eso, un tamaño de efecto convencional para una tabla 2x2 de baja frecuencia. Para NAS no se calculó un contraste formal. La auditoría del posttest se obtuvo en una pasada suplementaria del 12/09/2026, sin volcados exportados y sin registro del estado del código respecto de la corrección de concurrencia (sección 6.2). Las auditorías cubrieron la totalidad de los 17 productos del catálogo por condición (no una muestra de 5) y el stock registrado coincidió con el esperado en el 100% de los casos en ambas condiciones, sin variación que contrastar. No se aplicó corrección por comparaciones múltiples entre las tres variables: cada una responde a una pregunta distinta y solo TPP fue objeto de un contraste de hipótesis formal, por lo que dicha corrección no resulta aplicable en este caso.

La Tabla 5.1 reporta media y DE; la mediana, el rango y los cuartiles de TPP —estadísticos más robustos ante los valores atípicos observados, ver Figura 5.1— la complementan. En el pretest: mediana = 32,00 s (rango [25; 103] s; RIC [28,75; 38,00] s). En el posttest: mediana = 21,50 s (rango [17; 58] s; RIC [20,75; 24,25] s).

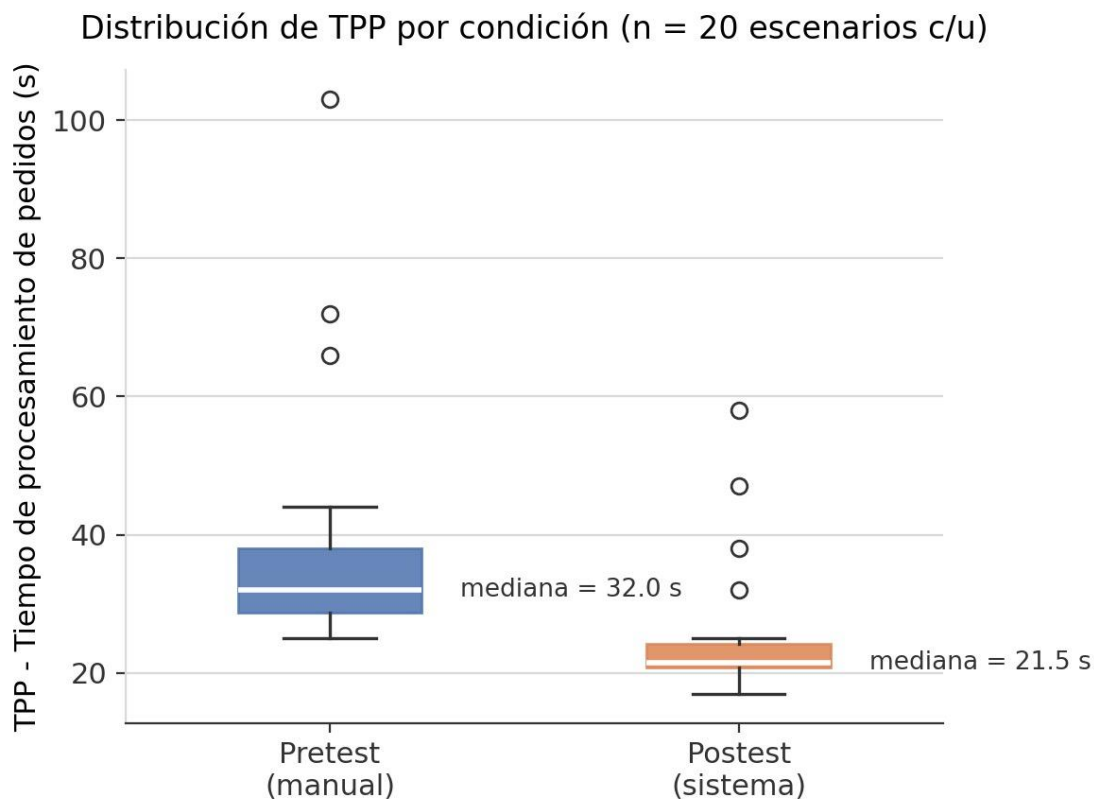


Figura 5.1. Diagrama de cajas de TPP por condición (n = 20 escenarios por condición). Los círculos representan valores atípicos; la línea blanca dentro de cada caja indica la mediana.

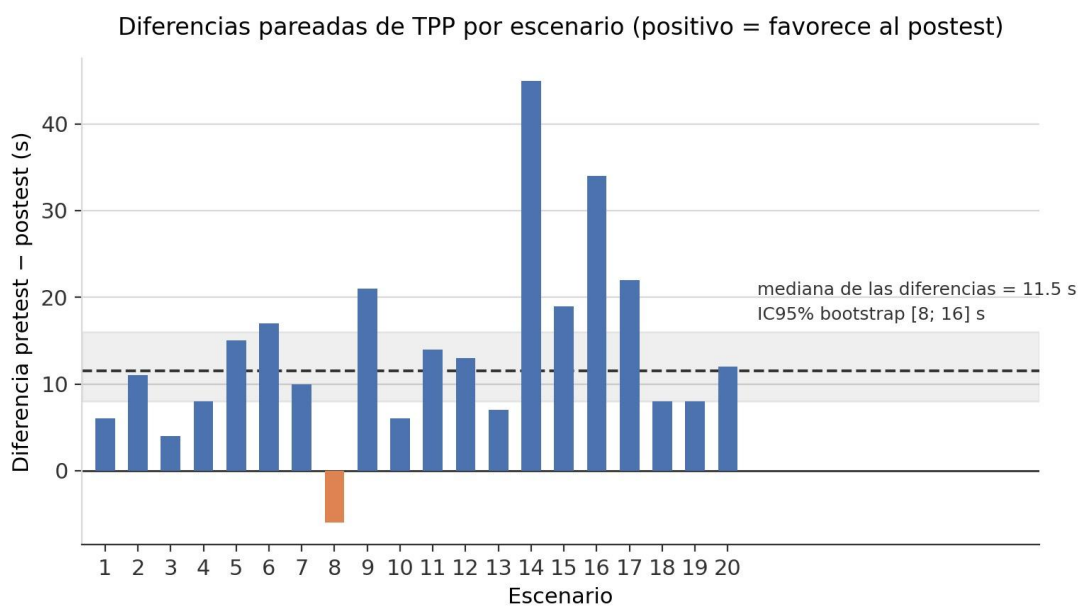


Figura 5.2. Diferencias pareadas de TPP (pretest - posttest) por escenario. La barra naranja (escenario 8) es el único caso que favorece al pretest. La línea punteada y la banda sombreada indican la mediana de las diferencias y su intervalo de confianza bootstrap del 95%.

5.2 Síntesis de hallazgos

De las tres variables definidas en la sección 3.5, TPP mostró la mejora más marcada y estadísticamente más sólida. El tiempo promedio de procesamiento de pedidos cayó de 39,30 s (DE = 19,51; mediana = 32,00 s) en la condición pretest a 25,60 s (DE = 10,52; mediana = 21,50 s) en la posttest: una reducción del 34,9% sobre las medias, por debajo del umbral del 50% fijado en H1. La mediana de las diferencias pareadas fue de 11,50 s, equivalente al 35,9% de la mediana del pretest; el intervalo de confianza bootstrap al 95% de esa reducción relativa, [25,8%; 44,4%], excluye el umbral del 50%, de modo que el rechazo del umbral de H1 no descansa solo en la cifra puntual. La reducción de las medianas de cada condición, de 32,00 s a 21,50 s, es del 32,8% (IC95% [24,1%; 40,8%]). Diecinueve de los veinte escenarios mostraron mejora en el posttest; uno (el escenario 8, en el que un reintento de selección de categoría alargó el posttest) favoreció al pretest (Figura 5.2). El escenario 14 del posttest (58 s, el valor máximo de esa condición) se abortó por un error al cargar la cantidad y se volvió a simular: el intervalo registrado corresponde solo a la resimulación, lo que favorece al posttest, y su efecto se acota en la sección 6.2. La prueba de Shapiro-Wilk sobre las diferencias pareadas rechazó el supuesto de normalidad ($W = 0,892$; $p = 0,029$), por lo que, conforme a lo previsto en la sección 3.4, el contraste se realizó con la prueba no paramétrica de Wilcoxon en lugar de la prueba t de Student. El resultado exacto ($p = 9,54 \times 10^{-6}$) confirma que la diferencia es estadísticamente significativa. El tamaño del efecto correspondiente ($r = 0,60$, con $N = 40$ observaciones totales, Rosenthal, 1991) se ubica igualmente en el rango convencionalmente considerado grande (Cohen, 1988). El intervalo de confianza bootstrap al 95% para la mediana de las diferencias pareadas ([8; 16] segundos), además, no incluye el cero, lo cual refuerza la robustez del hallazgo más allá del valor p, aun cuando la magnitud observada no alcance el umbral del 50% hipotetizado a priori.

TED, por su parte, no mostró diferencia entre condiciones. 2 de las 17 transacciones no vinculadas a stock insuficiente tuvieron una incidencia de operación que obligó a rehacer un paso, tanto en el pretest (escenarios 5 y 9, errores de registro en la planilla) como en el posttest (escenarios 8 y 14, errores de selección del operador): 11,8% en ambas condiciones (prueba de McNemar exacta, $p = 1,000$; detalle en el Anexo A). Como TED se reporta de forma descriptiva y sin hipótesis asociada (secciones 1.2 y 3.1), este resultado no confirma ni refuta ningún umbral; se discute en la sección 6.1.

NAS tampoco mostró diferencia entre condiciones: 100% de concordancia entre el stock registrado y el esperado según el diseño de escenarios, tanto en el pretest como en el posttest (17 de 17 productos auditados en cada condición). Al igual que TED, NAS se reporta de forma descriptiva, sin un umbral fijado a priori; este resultado se discute en la sección 6.1.

En síntesis: H1 no se confirma en su umbral literal del 50%, pero la reducción observada en TPP es estadísticamente significativa y de tamaño de efecto grande; TED y NAS, ambas descriptivas, no muestran diferencia entre condiciones. El hallazgo central de este trabajo es, entonces, una mejora real y sustantiva en el tiempo de procesamiento de pedidos, sin evidencia, en este diseño, de una mejora acompañante en la tasa de errores o en la exactitud del stock.

CAPÍTULO 6: Discusión

6.1 Interpretación de los resultados

La reducción observada en TPP (34,9% sobre las medias; 35,9% para la mediana de las diferencias pareadas respecto de la mediana del pretest, IC95% [25,8%; 44,4%]) es menor que el orden de magnitud que Davenport y Kirby (2016) atribuyen a la automatización de tareas administrativas estructuradas. Tampoco alcanza el umbral del 50% adoptado en la sección 3.1 para H1, y el intervalo de confianza de esa reducción excluye dicho umbral. No obstante, la mejora es consistente en dirección con el argumento de Moeuf et al. (2020) y de Sufi (2023) sobre el potencial de impacto de las soluciones de bajo costo y fácil integración —incluidas las plataformas low-code/no-code como n8n— en PyMEs. El sistema desarrollado no requirió infraestructura compleja y, aun así, produjo una reducción de tiempo estadísticamente significativa y de tamaño de efecto grande ($r = 0,60$; Cohen, 1988). Cabe señalar que el diseño adoptado carece de grupo de comparación no equivalente y se ubica, en sentido estricto, en la categoría de diseños preexperimentales (sección 3.4). Por eso, parte de esta reducción podría explicarse por fuentes de variación no controladas, en particular el efecto de práctica derivado de resolver los mismos veinte escenarios en ambas condiciones (secciones 3.9 y 6.2). La incorporación de escenarios de calentamiento (sección 3.4) mitigó parcialmente esta amenaza, sin eliminarla. Un caso concreto ilustra esta cautela: el escenario 8, el único que favoreció al pretest, correspondió a un contratiempo operativo puntual en el postest (un reintento de selección de categoría). Esto indica que el resultado agregado no está artificialmente libre de ruido en ninguna de las dos direcciones. Por ello, la reducción observada se interpreta como una estimación verificable y reproducible, obtenida bajo un instrumento de medición único y consistente entre condiciones (sección 3.5), y no como una cifra construida sobre datos no verificables. Debe leerse, sin embargo, como una estimación obtenida en condiciones controladas y con amenazas que en su mayoría favorecen a la condición automatizada (orden fijo de las corridas, mismos escenarios y práctica entre pasadas, sección 6.2), y por lo tanto como una cota superior probable del efecto en un comercio real.

El resultado de TED —sin diferencia entre condiciones (11,8% en ambas; McNemar exacto, $p = 1,000$)— contrasta con lo esperado a partir de la misma literatura. Esa literatura atribuye a la automatización una reducción en la tasa de errores humanos (Davenport y Kirby, 2016). La explicación más plausible no es que la automatización sea inefectiva para reducir errores, sino que el diseño experimental, tal como se advirtió en las secciones 1.2 y 3.1, no genera una tasa de error de base representativa de un operador real. Uno de los autores del sistema operó ambas condiciones, con pleno conocimiento del catálogo y del procedimiento; por eso, los errores registrados (dos por condición, detallados en el Anexo A) reflejan más bien la variabilidad esperable de una tarea manual repetitiva que una falla estructural de alguna de las dos modalidades. Un operador real, sin ese conocimiento ni ese entrenamiento previo, probablemente presentaría una tasa de error de base distinta —y potencialmente más alta en la modalidad manual— que este diseño no puede estimar. Además, la naturaleza de los errores difiere entre condiciones (de registro en la planilla en el pretest, de selección del operador en

el posttest; sección 3.5), de modo que la igualdad del 11,8% no compara exactamente la misma magnitud. En consecuencia, TED se reporta como indicador descriptivo, sin hipótesis asociada; su contrastación con operadores reales queda como pregunta abierta para la validación de campo (sección 7.2).

NAS tampoco mostró diferencia entre condiciones (100% en ambas). Corresponde precisar aquí el mecanismo real de actualización de stock del sistema, descrito en la sección 4.4. El descuento no ocurre de forma inmediata y automática al recibirse el pedido, sino cuando el administrador confirma el pedido desde el panel web (`PedidoService.cambiarEstado()`); en ese momento se ejecuta una operación atómica sobre la base de datos. Es decir, en la condición posttest la exactitud del stock también depende de una acción del operador: confirmar el pedido en el panel. Esto es análogo a como, en la condición pretest, depende de que el operador transcriba correctamente el descuento a la planilla. La diferencia entre ambas modalidades no es, entonces, manual con intervención humana versus automático sin intervención humana, sino el grado de acoplamiento y atomicidad de esa intervención. En el panel, confirmar el pedido y descontar el stock son una única operación atómica a nivel de base de datos; en la planilla, en cambio, son dos pasos independientes (registrar el pedido y actualizar el stock) que el operador puede omitir o ejecutar de forma inconsistente. Esta distinción teórica no se tradujo, en esta corrida, en una diferencia medible de NAS, y el diseño no podía producirla. El experimento es una simulación, sin mercadería ni un inventario físico independiente del registro: el stock esperado se calculó a partir del diseño de escenarios (sección 3.5). Por eso NAS compara, en el pretest, la aritmética manual del operador con la esperada y, en el posttest, el estado de la base con esa misma aritmética. Es una verificación de consistencia interna, y no puede detectar la desactualización que en un comercio real proviene de mercadería que se mueve sin quedar registrada. Ese fenómeno, y la eventual ventaja práctica de la atomicidad, solo podrían observarse en una validación de campo con inventario físico y operadores reales (sección 7.2). Cabe agregar que, durante la re-ejecución del experimento, se identificó (pasada descartada del 11/09, sección 3.9) y se corrigió un defecto real de concurrencia en esa misma operación de confirmación: una confirmación duplicada casi simultánea (por ejemplo, un doble clic) podía descontar el stock dos veces. El defecto se corrigió mediante una transacción atómica condicional a nivel de base de datos (sección 4.6). Aunque no forma parte de las variables medidas, este hallazgo respalda la afirmación anterior sobre la importancia de la atomicidad como mecanismo de integridad del stock, más allá de si el registro es manual o automatizado.

6.2 Limitaciones del estudio

Además de la limitación de validez externa señalada en la sección 3.9 —el experimento se ejecutó en un entorno controlado y no en un comercio real en operación— y de la precisión terminológica sobre la naturaleza preexperimental del diseño (sección 3.4), la ejecución concreta del experimento reveló limitaciones adicionales no anticipadas en el diseño original. Se detallan a continuación:

En primer lugar, el diseño intrasujeto ejecutó los mismos 20 escenarios de medición en un orden fijo: primero la modalidad manual (10/09/2026), luego la automatizada (12/09/2026), con más de un día de intervalo entre el cierre de una y el inicio de la otra. No hubo contrabalanceo, y el mismo operador (uno de los autores del sistema) intervino en ambas corridas. Para atenuar el efecto de práctica se incorporaron 5 escenarios de calentamiento no cronometrados antes de cada condición (sección 3.4). Esta incorporación reduce, pero no permite aislar cuantitativamente, la magnitud del efecto de práctica residual sobre los 20 escenarios de medición, que siguen siendo idénticos entre condiciones. A diferencia de versiones anteriores de este trabajo, TPP se midió en ambas condiciones con un instrumento único: grabación de pantalla con el reloj del sistema visible, leído al segundo (sección 3.5). Por eso, la asimetría instrumental ya no es una amenaza aplicable. La magnitud del efecto observado ($r = 0,60$) es compatible con una mejora real más allá del ruido de medición. Sin embargo, dado el diseño preexperimental, no puede descartarse por completo una contribución del efecto de práctica residual a la reducción del 34,9%-35,9% informada en la sección 5.2.

Se realizó, no obstante, un análisis exploratorio parcial para acotar el aprendizaje dentro de cada corrida cronometrada (docs/analisis/compute_practica_residual.py, Anexo B). Dado que los 5 escenarios de calentamiento no se cronometraron (sección 3.4), no es posible compararlos directamente contra los escenarios medidos. En su lugar, se comparó la primera mitad (escenarios 1 a 6) contra la segunda (escenarios 8 a 13) del subconjunto más homogéneo de escenarios medidos: los trece de tipo “Simple 1 producto” (Tabla 3.1). En ellos la complejidad de la tarea se mantiene aproximadamente constante, de modo que una tendencia decreciente resultaría atribuible principalmente a la práctica y no a diferencias de dificultad. En la corrida manual no se observó tendencia. La diferencia entre mitades fue de 1,50 s (32,33 s vs. 30,83 s) y la correlación entre la posición del escenario y su TPP no fue significativa (Spearman $p = -0,22$; $p = 0,47$; Pearson $r = -0,14$; $p = 0,64$). En la corrida automatizada, en cambio, sí se observó una tendencia decreciente: la correlación fue significativa con el estadístico no paramétrico ($p = -0,64$; $p = 0,02$) y no con el paramétrico ($r = -0,41$; $p = 0,16$). La diferencia entre mitades fue de 0,50 s (22,17 s vs. 21,67 s). Esa diferencia está atenuada por el escenario 8, el único que favoreció al pretest por un reintento de selección de categoría (sección 5.2). Es un valor alto (32 s), situado en la octava posición y por lo tanto en la segunda mitad: debilita la tendencia en lugar de producirla. Se lo excluyó entonces como variante de sensibilidad, definida a posteriori y aplicada a ambas condiciones. Sin ese escenario, la asociación se refuerza y es significativa con los dos estadísticos ($p = -0,78$; $p = 0,003$; $r = -0,77$; $p = 0,003$). La diferencia entre mitades asciende a 2,57 s (22,17 s vs. 19,60 s), frente a 0,53 s en la corrida manual. En síntesis, dentro de la corrida automatizada se observa una tendencia decreciente compatible con un efecto de práctica intra-corrida. Su magnitud, medida entre las mitades de una misma corrida, es pequeña frente a la reducción global de 13,70 s (menos de una quinta parte de ella). Dentro de la corrida manual no se observa. Este hallazgo debe distinguirse de la amenaza identificada en la sección 3.9, que es el arrastre entre condiciones: que el postest se beneficie de haber resuelto los mismos veinte escenarios en pasadas anteriores. Una tendencia intra-corrida, plana o marcada, no mide ese arrastre. Los datos disponibles permiten una cota indicativa, no una estimación: el intento de postest descartado del 11/09 (sección 3.9) fue una primera pasada completa del circuito automatizado sobre los

mismos escenarios, y sus tiempos fueron mayores que los del posttest definitivo (media de 31,20 s frente a 25,60 s; diferencia de 5,60 s; Wilcoxon pareado exacto $p = 5,3 \times 10^{-4}$; Tabla 6.1). Esa diferencia, equivalente al 41 % de la reducción global de 13,70 s, es compatible con un efecto de práctica entre pasadas del mismo circuito, aunque también podría deberse a otras causas (carga del servidor, latencia de la conexión, orden de los escenarios: en el intento descartado los escenarios 15 y 16 se ejecutaron en orden inverso). Por lo tanto, no puede descartarse que una parte relevante de la reducción de 13,70 s entre el pretest y el posttest definitivo se deba a la práctica y no al sistema. Contra el intento descartado, la reducción de las medias es del 20,6 % (Wilcoxon exacto $p = 8,3 \times 10^{-4}$). El arrastre entre condiciones se mantiene como limitación no estimada con precisión: su estimación completa, con escenarios de calentamiento cronometrados y contrabalanceo del orden, queda como tarea pendiente para una réplica futura del experimento (sección 7.2).

Pasada	Media (s)	Mediana (s)	Reducción de medias vs. pretest	Wilcoxon exacto vs. pretest	Escenarios que favorecen al pretest
Pretest manual (10/09)	39,30	32,00	—	—	—
Posttest descartado (11/09)	31,20	26,00	20,6 %	$p = 8,3 \times 10^{-4}$	3
Posttest definitivo (12/09)	25,60	21,50	34,9 %	$p = 9,5 \times 10^{-6}$	1

Tabla 6.1. TPP de las tres pasadas del experimento ($n = 20$ escenarios por pasada). Los valores p son exactos (Wilcoxon pareado; permutación de signos con rangos promedio). La diferencia entre el posttest definitivo y el descartado es de 5,60 s ($p = 5,3 \times 10^{-4}$). Fuente: docs/analisis/compute_intento_descartado.py (Anexo B).

En segundo lugar, el tamaño muestral se fijó en $n=20$ por condición (sección 3.4), por restricciones de tiempo del equipo. Esto limita la potencia estadística del contraste, aunque no impidió detectar el efecto grande observado en TPP.

En tercer lugar, quien operó la condición pretest es coautor del sistema: conoce el catálogo de productos y los procedimientos de carga con un nivel de familiaridad muy superior al de un operador real del comercio. Esto probablemente subestima la tasa de error de la modalidad manual y explica, al menos parcialmente, por qué TED no mostró diferencia entre condiciones (sección 6.1).

En cuarto lugar, las auditorías de stock (NAS) se realizaron en una única sesión al cierre de cada corrida completa, auditando la totalidad de los 17 productos del catálogo, en lugar de en puntos de control distribuidos a lo largo de la ejecución. Esto es consistente con la definición de NAS de la sección 3.5, pero no permite observar cómo evoluciona la desactualización del stock durante la corrida, solo su estado final. En el posttest, además, la auditoría no se hizo al cierre

de la corrida cronometrada: un reinicio accidental de la base de datos, antes de exportar el estado final, impidió conservar los datos de inventario de esa corrida. La auditoría se realizó entonces en una pasada suplementaria, el 12/09/2026, posterior a la corrida cronometrada, sobre los mismos 20 escenarios y sin video ni cronometraje. El defecto de doble descuento de stock ante confirmaciones duplicadas casi simultáneas se detectó en la pasada descartada del 11/09/2026 (sección 3.9) y su corrección se consolidó en el commit 97ce027 (13/09/2026). No se conserva un registro que permita establecer si esa corrección ya estaba aplicada durante la corrida cronometrada del 12/09 o se aplicó después: el árbol de trabajo de esa corrida no estaba commiteado. En consecuencia, no puede afirmarse que la corrida cronometrada y la pasada suplementaria se ejecutaron sobre el mismo código, y el 100% informado en la Tabla 5.1 debe leerse con esa salvedad. El defecto afectaba al descuento de stock ante confirmaciones duplicadas casi simultáneas, no a la recepción del pedido por el bot. Tampoco se exportaron volcados de stock de esa pasada, por lo que la fila NAS del posttest no tiene respaldo exportable, a diferencia del pretest, cuyo registro completo se conserva (ledger y volcados de stock). Ambas circunstancias se declaran como limitaciones de esa fila.

En quinto lugar, el escenario 14 del posttest se abortó y se volvió a simular. La planilla registra un error al cargar la cantidad de productos, por lo que el pedido se canceló antes de confirmarlo para no descontar stock. El intervalo registrado (21:16:55 a 21:17:53, 58 s) cubre solo la resimulación. El log de n8n muestra las 15 ejecuciones del intento abortado entre las 21:15:29 y las 21:16:43, fuera de ese intervalo, una de ellas con error; las 14 ejecuciones de la ventana registrada son coherentes con un pedido de tres productos. Omitir el intento abortado favorece a la condición posttest. Para acotar su efecto se recalculó el contraste incluyéndolo, con inicio en la primera ejecución de n8n (144 s) o, como cota superior, en el fin del escenario anterior (156 s) (compute_stats.py, Anexo B). La reducción de las medias baja del 34,9% a entre el 22,4% y el 23,9%, y la mediana de las diferencias, a 10,50 s. El resultado sigue siendo significativo (Wilcoxon exacto, $p = 0,0012$) y el intervalo de confianza de la reducción relativa, [23,3%; 42,2%], sigue excluyendo el umbral del 50%. El instante exacto de inicio del intento abortado solo puede leerse del video. Los resultados principales se informan con los datos tal como se registraron, y la variante con el intento abortado se presenta como análisis de sensibilidad. Cabe notar que el mismo incidente se cuenta como error en TED, mientras que su costo temporal queda fuera del intervalo de TPP en el resultado principal; la variante de sensibilidad corrige esa asimetría y acota el efecto de la decisión (22,4–23,9% frente a 34,9%).

En sexto lugar, los tiempos se leyeron sobre las grabaciones sin ocultar la condición y por un autor del sistema: identificar el instante de recepción y el de registro confirmado conlleva un margen de juicio que este diseño no controla. Se mitiga con la lectura del reloj al segundo y, solo para el posttest, con la corroboración de máquina del registro de ejecuciones de n8n (Anexo A); el pretest, al ser manual, no tiene ese respaldo.

Por último, el escenario de carrito vacío (Anexo A) se excluyó del análisis cuantitativo por no tener un equivalente válido en la modalidad manual. Su verificación exploratoria confirmó que exponía una excepción no controlada en el backend, corregida en el commit 1d95abf del 11/08/2026 (sección 4.6).

El dataset crudo del experimento, el flujo de automatización de n8n, el diagrama entidad-relación de la base de datos, capturas de la interfaz y del bot, y el enlace al repositorio de código se incluyen como anexos (véase Anexos). Esto garantiza la reproducibilidad de los resultados presentados. Esto no sustituye, sin embargo, a la validación del sistema en un comercio real en funcionamiento, que se mantiene como línea de trabajo futuro prioritaria (sección 7.2).

CAPÍTULO 7: Conclusiones y Trabajos Futuros

7.1 Conclusiones

OE1 (arquitectura cliente-servidor) y OE2 (API REST en Java con operaciones CRUD) se alcanzaron plenamente: el sistema implementado, descrito en las secciones 4.1 y 4.2, responde a ambos objetivos y fue el artefacto sobre el que se ejecutó el experimento, con la salvedad declarada en la sección 6.2: no se conserva registro de si la corrección de la confirmación de pedidos ya estaba aplicada durante la corrida cronometrada.

OE3 (interfaz de usuario en JavaScript) se alcanzó: la SPA descrita en la sección 4.1.1 cubre la gestión de productos, clientes, pedidos y stock, tal como se documenta en las capturas del Anexo E.

OE4 (bot de Telegram) y OE5 (automatización con n8n) se alcanzaron. La sección 4.4 describe el flujo de recepción y registro automatizado de pedidos, verificado además experimentalmente mediante los 20 escenarios posttest y documentado en el flujo exportado del Anexo C.

OE6 (evaluar el desempeño mediante TPP, TED y NAS) se alcanzó: se completó la medición de las tres variables (sección 5.1). TPP mostró una reducción real, estadísticamente significativa y de tamaño de efecto grande, aunque por debajo del umbral hipotetizado; TED y NAS, medidas de forma descriptiva, no mostraron diferencia entre condiciones, por las razones discutidas en la sección 6.1.

Respecto de la hipótesis de trabajo, H1 no se confirma en su umbral literal de una reducción de al menos el 50% del TPP: el valor observado fue del 34,9% sobre las medias y del 35,9% para la mediana de las diferencias pareadas respecto de la mediana del pretest (IC95% [25,8%; 44,4%], que excluye el umbral). Con la variante que incluye el intento abortado del escenario 14, la reducción de las medias baja al 22,4–23,9% y el intervalo de la reducción relativa sigue excluyendo el umbral (sección 6.2). La reducción es, no obstante, estadísticamente significativa ($p = 9,54 \times 10^{-6}$) y de tamaño de efecto grande ($r = 0,60$; Cohen, 1988), y se sostiene bajo un instrumento de medición único y consistente entre condiciones (sección 3.5). TED y NAS, reportadas de forma descriptiva y sin hipótesis asociada, no mostraron diferencia entre condiciones. Esto se debe a limitaciones del diseño: la condición pretest, operada por uno de los autores del sistema, no logró generar una tasa de error de base representativa de un operador real, y el experimento, al no contar con un inventario físico independiente del registro, no podía generar desactualización de stock. Estos resultados, por lo tanto, no deben interpretarse como evidencia de que la automatización no mejora esos aspectos, sino como una limitación de este diseño experimental. Su contrastación con operadores reales queda planteada como línea de trabajo futuro (sección 7.2).

Lo demostrado empíricamente por este trabajo es que el sistema desarrollado reduce, de forma estadísticamente significativa y con un tamaño de efecto grande, el tiempo de procesamiento de pedidos. Esto se comprobó en un entorno de prueba controlado, cuyos datos y artefactos de soporte se documentan íntegramente en los Anexos. Lo sugerido, pero no demostrado, es que también podría mejorar la exactitud del stock registrado y reducir la tasa de errores de carga en condiciones reales de operación, con operadores ajenos al equipo de desarrollo. Lo que queda abierto es precisamente esa pregunta, junto con el desempeño del sistema ante la variabilidad de un comercio real en funcionamiento, que se retoma en la sección 7.2.

7.2 Trabajos futuros

Esta sección reúne únicamente lo que queda pendiente; las mejoras técnicas ya realizadas sobre el artefacto se describen en la sección 4.6. La principal línea de continuidad identificada

es la validación de campo del sistema en un comercio real en operación, tal como se plantea en las secciones 1.6 y 3.9. Esto responde a que el presente trabajo evalúa el artefacto en un entorno controlado y no en condiciones reales de uso. Esa validación de campo es también la vía para contrastar TED y NAS con una tasa de error de base representativa de un operador real y con un inventario físico independiente del registro, que el diseño actual no permite generar (secciones 6.1 y 7.1).

Adicionalmente, se identifican como posibles extensiones: la integración con sistemas contables externos; la gestión de múltiples sucursales; la incorporación de módulos de facturación electrónica y de gestión de proveedores (conforme a lo señalado en la sección 1.6); la evaluación de la escalabilidad del sistema ante un mayor volumen concurrente de pedidos; y la incorporación de un token anti-CSRF (Anexo F). También queda pendiente una réplica del experimento que estime el arrastre entre condiciones (sección 6.2), con escenarios de calentamiento cronometrados y, de ser posible, con contrabalanceo del orden de las condiciones.

Referencias Bibliográficas

Las referencias se presentan de acuerdo con las normas de la American Psychological Association (APA), 7.^a edición.

- Adamopoulou, E., y Moussiades, L. (2020). An overview of chatbot technology. In I. Maglogiannis, L. Iliadis, y E. Pimenidis (Eds.), *Artificial intelligence applications and innovations. AIAI 2020. IFIP Advances in Information and Communication Technology* (Vol. 584, pp. 373–383). Springer. https://doi.org/10.1007/978-3-030-49186-4_31
- Bogner, J., Kotstein, S., y Pfaff, T. (2023). Do RESTful API design rules have an impact on the understandability of Web APIs? *Empirical Software Engineering*, 28(6), Article 132. <https://doi.org/10.1007/s10664-023-10367-y>
- Campbell, D. T., y Stanley, J. C. (1963). *Experimental and quasi-experimental designs for research*. Rand McNally.
- CEPAL (Comisión Económica para América Latina y el Caribe). (2019). *Mipymes en América Latina: un frágil desempeño y nuevos desafíos para las políticas de fomento. Síntesis* (LC/TS.2019/20). CEPAL. <https://repositorio.cepal.org/handle/11362/44603>
- CEPAL (Comisión Económica para América Latina y el Caribe). (2021). *Tecnologías digitales para un nuevo futuro* (LC/TS.2021/43). CEPAL. <https://repositorio.cepal.org/handle/11362/46816>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Cohen, J. (1988). *Statistical power analysis for the behavioral sciences* (2nd ed.). Lawrence Erlbaum Associates.
- Date, C. J. (2004). *An introduction to database systems* (8th ed.). Pearson.
- Davenport, T. H., y Kirby, J. (2016). *Only humans need apply: Winners and losers in the age of smart machines*. Harper Business.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. <https://www.ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley.
- Fritsch, T. (2024). Chatbots: An overview of current issues and challenges. In K. Arai (Ed.), *Advances in information and communication. FICC 2024* (Lecture Notes in Networks and Systems, Vol. 919, pp. 84–104). Springer. https://doi.org/10.1007/978-3-031-53960-2_7
- Gao, S., Teh, P.-L., y Ho, H. H. P. (2026). Digital transformation and innovation in small and medium enterprises (SMEs): A systematic review and future research agenda. *Cogent*

Business & Management, 13(1), Article 2612775.
<https://doi.org/10.1080/23311975.2026.2612775>

Heizer, J., Render, B., y Munson, C. (2023). *Operations management: Sustainability and supply chain management* (14th ed.). Pearson.

Hevner, A. R., March, S. T., Park, J., y Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–106. <https://doi.org/10.2307/25148625>

Khantong, S., y Sriboonlue, P. (2026). Robotic Process Automation in Business Process Management: A Systematic Literature Review and an Integrated Framework. *Technologies*, 14(4), Article 225. <https://doi.org/10.3390/technologies14040225>

Lercher, A., Glock, J., Macho, C., y Pinzger, M. (2024). Microservice API evolution in practice: A study on strategies and challenges. *Journal of Systems and Software*, 215, Article 112110. <https://doi.org/10.1016/j.jss.2024.112110>

Li, Z., Wu, C., Li, J., y Yuan, Q. (2026). Chatbot research in the fields of business and information systems: A systematic review and bibliometric analysis. *Aslib Journal of Information Management*, 78(4), 988–1014. <https://doi.org/10.1108/AJIM-05-2024-0427>

Mehta, I. (2024, July 23). Telegram's userbase climbs to 950M, plans to launch app store. *TechCrunch*. <https://techcrunch.com/2024/07/23/telegrams-userbase-climbs-to-950m-plans-to-launch-app-store>

Moeuf, A., Lamouri, S., Pellerin, R., Tamayo-Giraldo, S., Tobon-Valencia, E., y Eburdy, R. (2020). Identification of critical success factors, risks and opportunities of Industry 4.0 in SMEs. *International Journal of Production Research*, 58(5), 1384–1400. <https://doi.org/10.1080/00207543.2019.1636323>

n8n. (2022). *Sustainable Use License (n8n Community License)*. <https://docs.n8n.io/n8n-community-license>

n8n. (2024). *n8n documentation: Workflow automation*. <https://docs.n8n.io>

Newman, S. (2019). *Monolith to microservices: Evolutionary patterns to transform your monolith*. O'Reilly Media.

OECD. (2023). *OECD SME and Entrepreneurship Outlook 2023*. OECD Publishing. <https://doi.org/10.1787/342b8564-en>

Parra-Sanchez, D. T., y Talero-Sarmiento, L. H. (2024). Digital transformation in small and medium enterprises: A scientometric analysis. *Digital Transformation and Society*, 3(3), 257–276. <https://doi.org/10.1108/DTS-06-2023-0048>

Pfister, P., y Lehmann, C. (2023). Measuring the success of digital transformation in German SMEs. *Journal of Small Business Strategy*, 33(1), 1–19. <https://doi.org/10.53703/001c.39679>

- Richardson, L., y Ruby, S. (2007). *RESTful web services*. O'Reilly Media.
- Rosenthal, R. (1991). *Meta-analytic procedures for social research* (Rev. ed.). Sage Publications.
- Sufi, F. (2023). Algorithms in low-code-no-code for research applications: A practical review. *Algorithms*, 16(2), 108. <https://doi.org/10.3390/a16020108>
- Telegram. (2024). *Telegram Bot API documentation*. <https://core.telegram.org/bots/api>
- van der Aalst, W. M. P. (2016). *Process mining: Data science in action* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-662-49851-4>

Anexos

Se incluyen a continuación los materiales de soporte que permiten verificar y reproducir los resultados presentados en este informe.

Anexo A. Dataset crudo del experimento

Archivo: `Plantilla_Experimento_Pretest_Postest.xlsx`

Contiene seis hojas. (1) Escenarios: descripción de los 20 escenarios de pedido estandarizados utilizados en ambas condiciones (sección 3.4). (2) Warmup: los 5 escenarios de calentamiento previos a cada condición, excluidos del análisis (sección 3.4). (3) Pretest: horarios de inicio y fin —leídos de la grabación de pantalla, sección 3.5— registrados para cada escenario bajo la modalidad manual, y el registro de errores de carga. (4) Posttest: la misma estructura para la modalidad automatizada, con la primera y la última ejecución de $n8n$ de la ventana de cada escenario. (5) Auditoria_Stock: el stock esperado según el diseño de escenarios (columna «Stock físico esperado», calculado como stock inicial menos consumo previsto) y el stock registrado (por el operador o por el sistema) sobre la totalidad de los 17 productos del catálogo, utilizados para calcular NAS. (6) Metadatos_corrida: las fechas, horarios, operadores, versiones de software y archivos de video de cada corrida. Sobre estos datos crudos se calcularon, de forma externa a la planilla, los estadísticos descriptivos e inferenciales presentados en la Tabla 5.1 (Anexo B). Las columnas de TPP y de coincidencia de stock se calculan en la planilla con fórmulas, cuyo resultado se guarda también en el archivo.

Corroboración del posttest. El registro de ejecuciones de $n8n$ (`docs/analisis/rerun/n8n_executions_postest.csv`, con marcas en UTC) corrobora, de forma independiente de la grabación de pantalla, los veinte tiempos del posttest. Las 153 ejecuciones de la corrida del 12/09/2026 (21:08:15 a 21:21:51, hora de Argentina) coinciden con la composición de cada escenario: 6 por escenario de un producto, 10 de dos productos, 14 de tres y 8 en el escenario 8, por el reintento de selección de categoría que la planilla registra. De ellas, 138 caen dentro de las ventanas declaradas y las 15 restantes corresponden al intento abortado del escenario 14 (sección 6.2). El cruce se realizó con `docs/analisis/cruzar_log_n8n.py` (salida en `docs/analisis/salida_cruzar_log_n8n.txt`), y `docs/analisis/rerun/README.md` identifica qué corrida documenta cada conjunto de datos. Esta corroboración cubre solo el posttest: el pretest es manual y no genera registro de máquina, por lo que sus veinte tiempos descansan únicamente en la grabación de pantalla. No se trata de una diferencia de instrumento, que es el mismo en ambas condiciones, sino de verificabilidad.

Referencia:

https://docs.google.com/spreadsheets/d/1YY23zZMBffX5gjKMN9Uy8w6rJ9jDVK1q/edit?usp=drive_link&ouid=114634369866028785057&rtpof=true&sd=true

Permisos de acceso: el enlace es de acceso público (cualquier persona que lo posea puede visualizar la planilla, sin necesidad de iniciar sesión ni de pertenecer a un dominio institucional).

Anexo B. Análisis estadístico

Los estadísticos reportados en la sección 5.1 —medias, desvíos estándar, prueba de Shapiro-Wilk, prueba de Wilcoxon, tamaño del efecto r y el intervalo de confianza bootstrap para la mediana de las diferencias pareadas de TPP, junto con el de la reducción relativa (sección 5.2)— se calcularon a partir de los datos crudos del Anexo A. Para ello se utilizó la biblioteca SciPy de Python. El script reproducible (`docs/analisis/compute_stats.py`) y la salida real de su ejecución (`docs/analisis/salida_compute_stats.txt`) se incluyen en el repositorio de código (Anexo F), de modo que los valores informados pueden recalcularse directamente sobre el Anexo A sin depender de una reconstrucción manual. El esquema de la base de datos (DDL), generado a partir de las seis entidades JPA descritas en la sección 4.3, se documenta en `docs/analisis/schema.sql` dentro del mismo repositorio. El análisis exploratorio del efecto de práctica residual discutido en la sección 6.2 se calculó de la misma manera, a partir de los horarios crudos de las hojas Pretest y Posttest del Anexo A, con el script `docs/analisis/compute_practica_residual.py` —que incluye una variante de sensibilidad sin el escenario 8— y su salida real `docs/analisis/salida_compute_practica_residual.txt`, incluidos también en el repositorio de código (Anexo F). El script `compute_stats.py` incluye además la estimación por intervalo de la reducción relativa (sección 5.2) y el análisis de sensibilidad del escenario 14 (sección 6.2), y el cruce del registro de $n8n$ contra las ventanas del posttest se realizó con `docs/analisis/cruzar_log_n8n.py` (Anexo A). La comparación con la pasada descartada del 11/09 (Tabla 6.1) se calculó con `docs/analisis/compute_intento_descartado.py`, a partir de la planilla archivada en `docs/analisis/rerun/intento1_posttest_bug_confirmacion_doble/`, y su salida se puede regenerar con el mismo comando.

Anexo C. Flujo de automatización de $n8n$

Archivo: `workflowN8N.json`

Exportación completa del flujo de trabajo implementado en $n8n$, con 31 nodos que cubren la totalidad del proceso descrito en la sección 4.4: disparador de Telegram; distinción entre mensajes de texto y `callback_query`; construcción de los menús de categorías, productos y cantidad; gestión del carrito; confirmación y cancelación del pedido; manejo del caso de stock insuficiente; y webhook de notificación de cambios de estado hacia el cliente.

Referencia: repositorio de código fuente (Anexo F), archivo `workflowN8N.json` en la raíz del proyecto. Es la misma fuente que se versiona junto con el backend, sin copias adicionales en otros servicios. El enlace de Google Drive que se adjunta a continuación reproduce ese mismo archivo; se ofrece únicamente como acceso alternativo para su visualización directa, sin necesidad de abrir el repositorio de GitHub.

Referencia:

https://drive.google.com/file/d/1QNbTc3_xuVtLR0x_DRRYvVw3qM5kZDuA/view?usp=drive_link

Permisos de acceso: el enlace de Google Drive es de acceso público (no requiere autenticación).

Anexo D. Modelo de datos

Archivos: `diagrama_arquitectura.png`, `diagrama_der.png`, `diagrama_clases.png`, `diagrama_secuencia.png`, `diagrama_bpmn_pedido.png` (repositorio de código, Anexo F, carpeta docs/diagramas/)

Diagrama de arquitectura del sistema, diagrama entidad-relación de la base de datos, diagrama de clases UML, diagrama de secuencia del flujo completo de pedidos y diagrama de proceso por carriles del flujo de toma de pedidos. Los cinco diagramas se presentan también en el cuerpo del informe (Figura 4.1, sección 4.1; Figuras 4.2 y 4.3, sección 4.3; Figuras 4.4b y 4.4c, sección 4.4). El diagrama entidad-relación y el diagrama de clases corresponden a las seis entidades descritas en la sección 4.3 (productos, categorías, pedidos, items_pedido, usuarios, administradores) y declaran los tipos de datos vigentes en las entidades JPA: `BigDecimal/DECIMAL(10,2)` para precios y totales, `LocalDateTime/DATETIME` para la fecha y hora del pedido, y `String/VARCHAR` para el teléfono. Cada uno confirma, desde su notación, que la relación entre usuarios y pedidos, planificada en el diseño, no se encuentra implementada al momento de este informe.

Referencia: repositorio de código fuente (Anexo F), carpeta docs/diagramas/.

Anexo E. Capturas de la interfaz (SPA) y del bot de Telegram

Nueve capturas de pantalla (numeradas del 1 al 9) documentan lo siguiente: la pantalla de login del panel de administración; el panel de control con acceso a Control de Stock y Pedidos Telegram; la gestión de categorías y productos (alta, edición y eliminación); la interacción completa con el bot de Telegram (selección de categoría, producto y cantidad mediante botones, confirmación y envío del pedido); la recepción del pedido en el panel de Pedidos Telegram y su confirmación desde la interfaz web; el caso de cancelación de un pedido por carrito vaciado desde el bot; y el historial de pedidos con sus filtros de búsqueda.

Referencia:

https://drive.google.com/drive/folders/1gv1ZH7xTSmKyMZwvJ69vLzcdClqsTjYU?usp=drive_link

Permisos de acceso: la carpeta es de acceso público (cualquier persona con el enlace puede visualizar las nueve capturas, sin necesidad de autenticarse).

Anexo F. Repositorio de código fuente

Repositorio: https://github.com/marianovidela1988-svg/integrador_final

Limitaciones de seguridad conocidas. La protección contra CSRF está deshabilitada (`csrf.disable()`) y el sistema no emite un token anti-CSRF; el riesgo se mitiga con el atributo `SameSite=Strict` de la cookie de sesión JWT. Se acepta como limitación para el alcance de este

trabajo, que evalúa el artefacto en un entorno controlado, y debería resolverse antes de una implementación real (sección 7.2).

Nota sobre herramientas de prueba de API

Conforme a lo indicado en la sección 4.5, el equipo no utilizó una colección de Postman ni una especificación OpenAPI/Swagger para probar los endpoints de la API. Las pruebas se realizaron consumiendo la API desde el propio frontend de la aplicación y, para los endpoints de toma de pedidos, mediante interacción directa con el bot de Telegram. Este punto se deja explícitamente señalado como no aplicable a la metodología de pruebas efectivamente empleada en este trabajo, en lugar de omitirse sin aclaración.